

wine_analysis

May 10, 2025

0.1 Vinho Verde Wine Quality Analysis Notebook

0.1.1 Notebook by: [Sofia French](#)

0.1.2 Student Number: 230057999

0.2 Table of contents

1. Introduction
2. The Problem Domain
3. The Datasets at a Glance
4. Task 1: Exploratory Analysis
5. Task 2: Variable Analysis
6. Task 3: Modelling
7. Task 4: Model Evaluation
8. Extensions Overview
9. Conclusion
10. References

0.3 Introduction

This notebook was developed for my **Data Science** project (2034) assigned in Stage 2 of my degree at Newcastle University. It follows the task description highlighted on Canvas and draws inspiration from public resource notebooks, such as [Randal S. Olson's](#) and all published under the

data_visulisation GitHub repository. Specifically, following the structure of these notebooks has helped me effectively tell my “*data science story*” in a readable and consistant way.

As submission files were decided independently, to demonstrate the work I’ve completed I’ve chosen to include most of my GitHub repository, **excluding**: a latex/ dir, it’s helper bin/ dir, and my Makefile**.

The latex/ (and bin/) directories contain drafts of task findings for my final report, and as **make** is not automatically installed on all machines (e.g. Windows) and thus has been excluded.

A brief overview of my file structure has been attatched in the root README and pre-generated docs have been included where necessary as a precaution, but can be overwritten.

0.4 The Problem Domain

As outlined in the provided task description, I have been “*hired to consult for a winery in Portugal*” who wish to understand the truth behind the **quality disparity** in their **Vinho-Verde** wines.

Provided are three datasets,

- winequality-red.csv
- winequality-white.csv
- winequality.names

The two .csv files include large data for red and white wines, and each contains data points with a set of 12 variables; 11 of which are physicochemical, with the last being the perceived quality rating.

My task is to create “*one or more models to predict quality given all other variables*”, in order for the winery to determine which properties of their wine are most influential to quality.

0.5 The Datasets at a Glance

0.5.1 Answering Olson’s Questions

[Randal S. Olson](#) mentioned in his notebook, “*the first step to any data analysis project is to define the question or problem we’re looking to solve, and to define a measure (or set of measures) for our success at solving that task*”. With that in mind, I examined the project and it’s requirements using [Olson’s](#) questions:

Did you specify the type of data analytic question?

I’m trying to predict the quality of Vinho Verde wines based on 11 properties, e.g. **acidity**, **sugar**, and **alcohol content**. This is both an exploratory and predictive modelling problem. First, I’ll need to explore and visualise how these features relate to a wines perceived quality, then I can build models to classify and regress the quality based on the most influential features.

Did you define the metric for success before beginning?

It differs based on whether I’m using classification or regression, but both should use **k-fold cross validation**.

For *classification* models, the suggested metrics are: 1. F1-score 2. AUC 3. ROC

For *regression* models: 1. RMSE 2. MAE 3. Related “error metrics”

Did you understand the context for the question and the scientific or business application?

I’m consulting for a winery in Portugal who wish to understand which features of their wines most strongly determine quality.

This analysis will inform the winery on which properties of wine are the most and least influential to quality so they can adjust their wine production process in order to make higher quality wines (at least, I assume, it’s proven difficult to get ahold of any stakeholders).

Did you record the experimental design?

In both datasets, each record corresponds to a single wine. As told to in the project report, the two datasets are large and “*each data point is a set of 12 variables; 11 of these are physicochemical measures, and the last is the perceived quality (sensory output)*”.

Did you consider whether the question could be answered with the available data?

The dataset is well suited for determining factors that influence wine quality. However, there are **limitations**: - Quality is subjective and differs on consumer basis - There may be other factors that influence quality that are not recorded, like aging

0.5.2 Exploring the Datasets

Before I begin with the required tasks, as suggested in [Olson’s](#) notebook and hinted at in the [Canvas](#), it’s best to quickly check the data’s sanity.

Specifically, I’m checking for errors or unexpected qualities in either dataset. For this project, both datasets are well regarded and tested, meaning it is safe to assume the data is clean. However, for good practice, it can’t hurt to double check.

To do this, I’ll read both datasets into a pandas data frame (*df*) and display the first 5 records. First, the red wine dataset:

```
[1]: import pandas as pd

wine_red = pd.read_csv("../datasets/winequality-red.csv", sep=";")
wine_red.head()
```

```
[1]:   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0           7.4             0.70         0.00           1.9       0.076
1           7.8             0.88         0.00           2.6       0.098
2           7.8             0.76         0.04           2.3       0.092
3          11.2             0.28         0.56           1.9       0.075
4           7.4             0.70         0.00           1.9       0.076

   free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0             11.0             34.0    0.9978  3.51       0.56
```

1	25.0	67.0	0.9968	3.20	0.68
2	15.0	54.0	0.9970	3.26	0.65
3	17.0	60.0	0.9980	3.16	0.58
4	11.0	34.0	0.9978	3.51	0.56

	alcohol	quality
0	9.4	5
1	9.8	5
2	9.8	5
3	9.8	6
4	9.4	5

As expected, the data is loaded and populated. Now for white wine:

```
[2]: wine_white = pd.read_csv("../datasets/winequality-white.csv", sep=";")
      wine_white.head()
```

```
[2]:   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides \
0           7.0           0.27           0.36           20.7         0.045
1           6.3           0.30           0.34            1.6         0.049
2           8.1           0.28           0.40            6.9         0.050
3           7.2           0.23           0.32            8.5         0.058
4           7.2           0.23           0.32            8.5         0.058
```

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	45.0	170.0	1.0010	3.00	0.45	
1	14.0	132.0	0.9940	3.30	0.49	
2	30.0	97.0	0.9951	3.26	0.44	
3	47.0	186.0	0.9956	3.19	0.40	
4	47.0	186.0	0.9956	3.19	0.40	

	alcohol	quality
0	8.8	6
1	9.5	6
2	10.1	6
3	9.9	6
4	9.9	6

Again, the dataset appears complete, with 12 fields and each as listed in the brief. No problems here.

Next, starting again with red wine, I'll use pandas `describe` to examine input data from two different key fields: `alcohol` and `residual sugar`.

```
[3]: wine_red[["alcohol", "residual sugar"]].describe()
```

```
[3]:   alcohol  residual sugar
count  1599.000000      1599.000000
mean    10.422983         2.538806
```

std	1.065668	1.409928
min	8.400000	0.900000
25%	9.500000	1.900000
50%	10.200000	2.200000
75%	11.100000	2.600000
max	14.900000	15.500000

Focusing first on **alcohol**: - It has a good range, with the **min** value at 8.4 and **max** value at 14.9. - This makes it clear why later separating into a three valued variable **alcohol_cat**, with *low*, *mid*, and *high* values would be beneficial - there is sufficient spread to justify three levels.

Next on **residual sugar**: - It appears most red wines are dry as the mean is approximately 3. - The data is skewed heavily to the right tail, as some outlier values have residual sugars as high as 15.5. - This makes it clear why a binary value splitting the sweetness evenly (**isSweet**) would be beneficial when examining this feature.

Moving onto white wine, I'll perform the same analysis.

```
[4]: wine_white[["alcohol", "residual sugar"]].describe()
```

```
[4]:
```

	alcohol	residual sugar
count	4898.000000	4898.000000
mean	10.514267	6.391415
std	1.230621	5.072058
min	8.000000	0.600000
25%	9.500000	1.700000
50%	10.400000	5.200000
75%	11.400000	9.900000
max	14.200000	65.800000

Focusing first on **alcohol**: - Very similar to red wine, however white wines have a slightly higher mean (10.51... vs. 10.42...). - Spread is also larger than red (stddev as 1.23... vs. 1.07...). - But again, this is a well distributed variable and thus supports separating into a three valued **alcohol_cat** variable.

Next on **residual sugar**: - Much higher mean than in the red wine dataset (mean = 6.39... vs. 2.54...). - The max value (65.8) is an extreme outlier. - Again, the distribution is very right tail heavy, though much more so than with the red wines. - The mean (6.39...) again suggests many dry wines and a lower quantity of sweet wines.

First Conclusions For both datasets: - In the later analysis with the **alcohol_cat** category, it would make sense to split into three values as both datasets have good spread (between **min** and **max** valued.) - In the later analysis with the **isSweet** category, it would make sense to split the data more evenly to avoid class imbalance: having a lack of high sweetness wines and a large amount of dry wines.

Since both datasets seem initially OK, I'll now create a data frame to contain them for later plotting and analysis, and store the colours and palettes for combined plots.

```
[5]: wine_red["type"] = "red"
      wine_white["type"] = "white"
      wine_all = pd.concat([wine_red, wine_white])

      palette_red_white = {"red": "#d62728", "white": "#1f77b4"}
```

First Plots Before I explore the two fields any further, in preparation for both initial and later plots, I'll import my chosen libraries: `matplotlib` and `seaborn`.

Both were chosen over the other mentioned alternative, `Altair` for the same reasons: 1. Simpler
2. More tutorials 3. Widely used in example notebooks

```
[6]: import matplotlib.pyplot as plt
      import numpy as np
      import seaborn as sb

      # Set style
      sb.set_theme(style="whitegrid", font_scale=1.2)
      palette_red = sb.color_palette("Reds", n_colors=10)
      palette_white = sb.color_palette("Blues", n_colors=10)
```

Next, I'll generate a scatterplot matrix for both datasets.

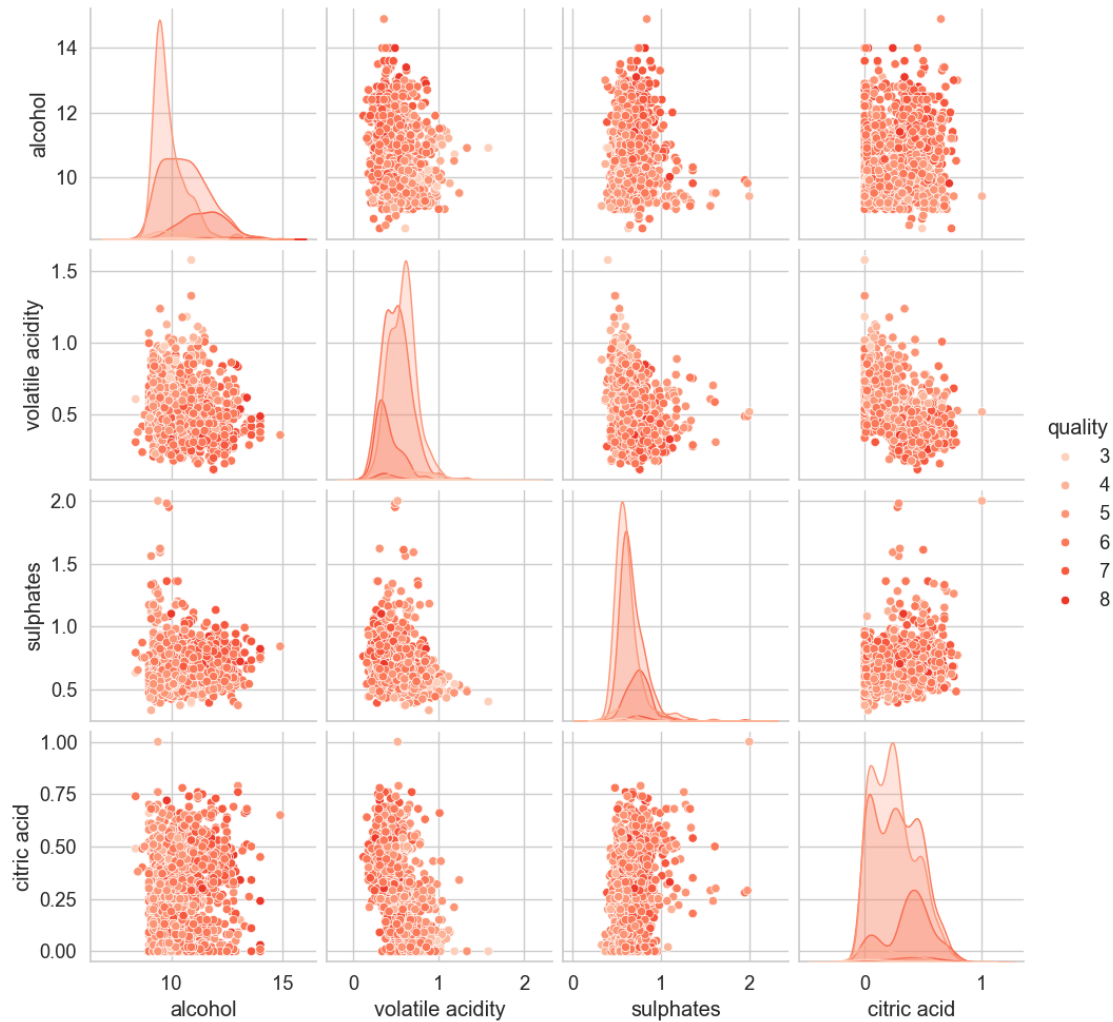
This was suggested in [Olson's](#) notebook as a tool to quickly check for errors or anomalies.

Originally, I compared every category against `quality`, but the 121 generated scatterplots were too busy for me to gain any insight from!

Instead, for a quick initial look, I'll compare `alcohol`, `volatile acidity`, `sulphates` and `citric acid` against each other. The shade of each point represents `quality` from a scale of 3-8.

For use in my task drafts, I'll save all generated figures for this project in the directory `figures/`. For more information on this, see Introduction.

```
[7]: sb.pairplot(
      wine_red[["alcohol", "volatile acidity", "sulphates", "citric acid", "quality"]],
      hue="quality",
      palette=palette_red[1:7]
    )
plt.savefig("../figures/init_scatterplot_matrix_red")
```



Observations A quick look at patterns in this plot suggests:

For Alcohol: - In every plot (excluding **volatile acidity**) where **alcohol** is compared, higher quality wines tend to correlate to higher alcohol content. - The KDE plot shows that low quality wines peak at **lower alcohol levels**, whereas high quality wines peak at **higher alcohol levels**.

For Volatile Acidity - The trend is the opposite to **alcohol** - higher **volatile acidity** appears to correlate with lower quality wines. - In the KDE, it seems wines with lower qualities cluster at higher values of **volatile acidity**.

For Sulphates - There's a slight pattern with high quality wines having slightly higher sulphates, but not strongly enough to apply causation.

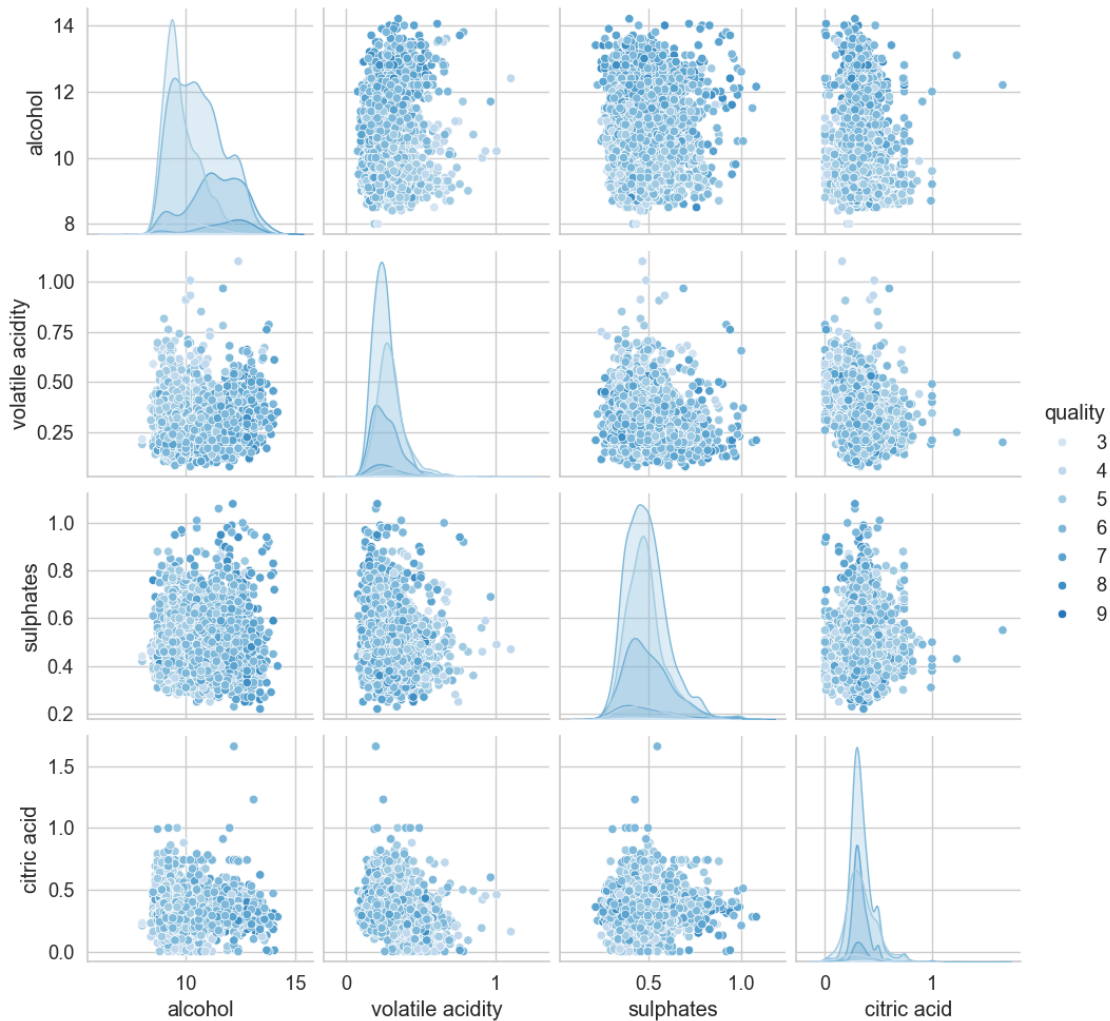
For Citric Acid - Hard to interpret — there's no obvious relationship and dots are scattered.

First Plot Conclusions

- **Alcohol** appears to be strongly correlated with better quality wines, and is the most predictive feature out of the few I plotted and interpreted.
- **Volatile acidity** is negatively correlated with quality.
- **Sulphates** may be slightly positively correlated with quality.
- **Citric acid** may not be a strong predictor.

Now to examine the *white wine dataset*:

```
[8]: sb.pairplot(
    wine_white[["alcohol", "volatile acidity", "sulphates", "citric acid", "quality"]],
    hue="quality",
    palette=palette_white[1:8]
)
plt.savefig("../figures/init_scatterplot_matrix_white")
```



Observations In the white wine scatterplot matrix:

For Alcohol: - Just like in red wine: higher alcohol tends to mean higher quality. - The KDE plot shows highest quality wines peak again around higher alcohol values.

For Volatile Acidity - Less dramatic than in red wine, but trend keeps - higher volatile acidity tends to correlate with lower quality wines. - In the KDE, low qualities are again concentrated at higher values.

For Sulphates - Again, there's some correlation: high quality wines could be influenced by higher sulphates. - KDE doesn't suggest much correlation.

For Citric Acid - Similar to red: no obvious relationship with quality. - KDE shows qualities are overlapping.

First Plot Conclusions

- **Alcohol** is a strong predictor of quality in both red and white wines, and may be a useful feature for later modelling.
- **Volatile acidity** is negatively correlated with quality, and again may be helpful later.
- **Sulphates** *could* have a slight positive relationship with quality.
- **Citric acid** is probably worth dropping in my later modelling.

0.5.3 Data Integrity Assumption

[Olson's](#) notebook has other steps before exploring his dataset, including **tidying** and **unit testing** the data.

I have chosen to skip these steps for a few reasons: - The red and white wine datasets used in this project are widely explored and arguably benchmark datasets with almost 200 results on Kaggle appearing for the search "vinho verde". - The datasets were provided by the university.

And given the fact that they contain: - No missing values - No unusual records - Contain the expected features

There is no need for extensive data cleaning or validation, though I will apply stripping to each column to ensure consistency when referencing variables.

```
[9]: wine_red.columns = wine_red.columns.str.strip()
     wine_white.columns = wine_white.columns.str.strip()
```

0.6 Task 1: Exploratory Analysis

Now the data has been *loaded*, *checked*, *plotted* and *stripped*, it's time to move onto the required tasks.

Briefly, for Task 1, the requirements are:

- (a) Plot and compare the distribution of wine **quality** for red and white wines.
- (b) Create a 3 level **alcohol_cat** variable (low, mid, high) based on **mean** $\pm$ **stddev**.
- (c) Analyse and plot wine quality distribution by **alcohol** to assess its impact on quality.

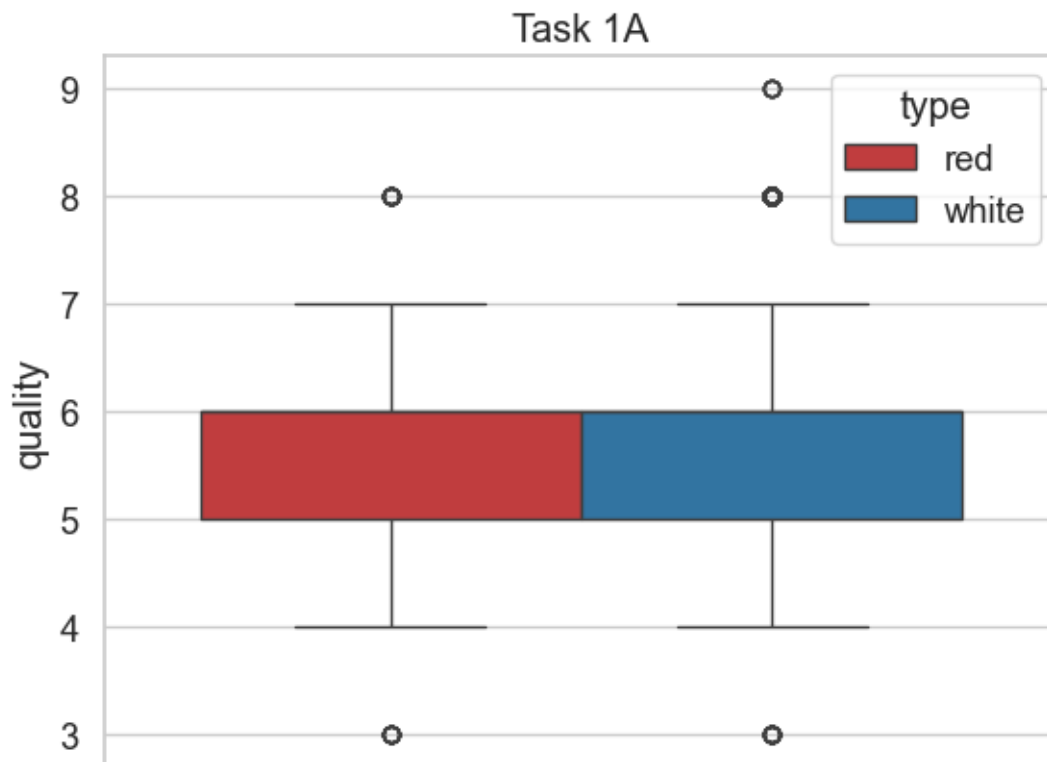
- (d) Create a binary `isSweet` variable by choosing a `residual_sugar` threshold that balances the classes.
- (e) Compare wine quality distributions for sweet vs dry wines to see if sweetness affects perceived quality.

0.6.1 Task 1A: Wine Quality Distribution

To begin, I'll start with Task 1A: plotting and comparing how wine quality differs between red and white wines.

For this task, I'll first use a box plot, as `quality` is discrete, and then a violin plot to examine the effect of class distributions.

```
[10]: sb.boxplot(  
    hue="type",  
    y="quality",  
    data=wine_all,  
    palette=palette_red_white  
)  
plt.title("Task 1A")  
plt.savefig("../figures/task1a_box_both")  
plt.show()
```



Box Plot Observations In the box plot above, for red and white wines:

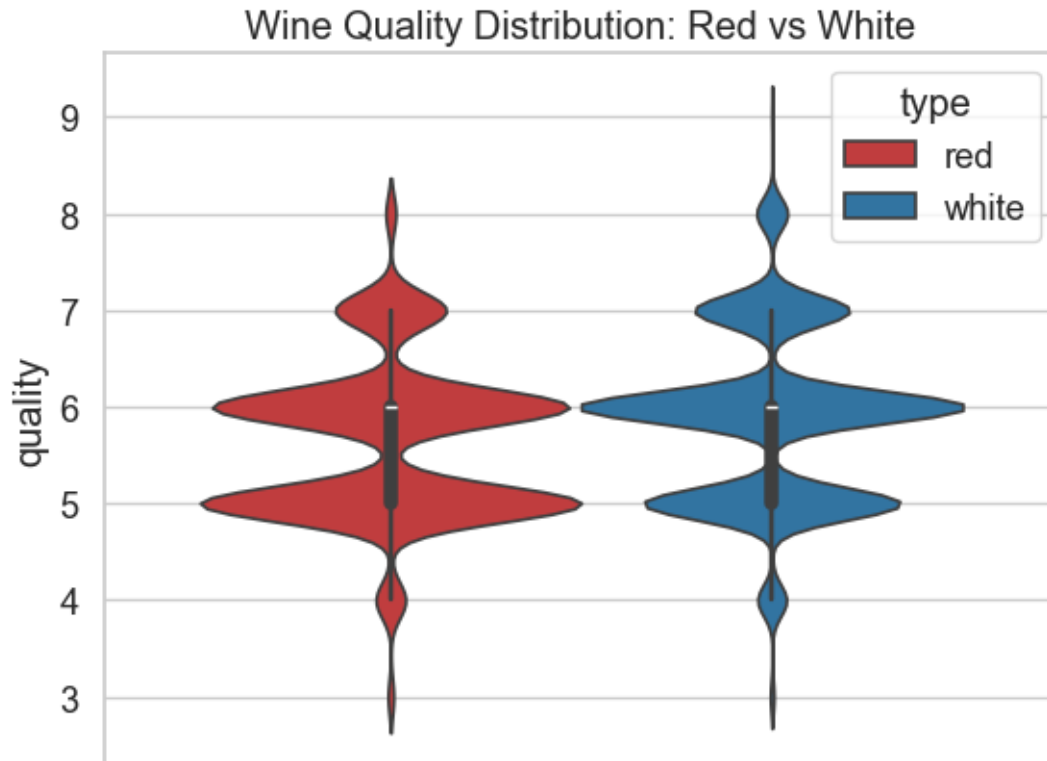
Statistic	Red Wine	White Wine
Median	5-6	5-6
IQR	5-6	5-6
Range	4-7	4-7
Outliers	3,8	3,8,9

The plot shows: - Central tendency is the same — both reds and whites cluster around a median of 5-6. - Spread is nearly identical — the IQR is the same for both. - White wines show a slightly longer upper tail — suggesting highest quality wines are more common in white wines than reds.

Box Plot Conclusion White and red wines have very similar quality distributions, but white wines may have a slightly higher upper potential for quality.

Violin Plot Now lets plot the same data but this time using a **violin plot**.

```
[11]: sb.violinplot(  
    hue="type",  
    y="quality",  
    data=wine_all,  
    palette=palette_red_white,  
    inner="box",  
    linewidth=1.2  
)  
  
plt.title("Wine Quality Distribution: Red vs White")  
plt.savefig("../figures/task1a_violin_both")  
plt.show()
```



Violin Plot Observations The plot shows: - The mode for *both* datasets is 6, with 5 and 6 quality densities being almost identical for red wines. - White wines have more spread between 5 and 7, and higher density at 7 and 8. - A tiny peak at 3 is visible for red wine, but not for white. - White wine has a longer tail, confirming the highest quality wines are in the white wines and that red wines dataset doesn't contain any qualities higher than 8.

Violin Plot Conclusion White and red wines have very similar quality distributions, but white wines have a slightly larger distribution and longer tail, confirming it holds more records and higher quality wines.

0.6.2 Task 1B: Discretise Alcohol Content

Next is Task 1B: *"discretise the alcohol content variables into low, mid, and high based on its distribution"*.

To complete this task, following the description, I need to create a three valued variable `alcohol_cat` that represents:

- **low** alcohol $< (\text{mean} - \text{stddev})$
- $(\text{mean} - \text{stddev}) < \text{mid}$ alcohol $< (\text{mean} + \text{stddev})$
- **high** alcohol $> (\text{mean} + \text{stddev})$

```
[12]: def add_alcohol_category(df):
    mean = df["alcohol"].mean()
    stddev = df["alcohol"].std()

    def discretise(alcohol):
        if alcohol < mean - stddev:
            return "low"
        elif alcohol > mean + stddev:
            return "high"
        else:
            return "mid"

    return df.assign(alcohol_cat=df["alcohol"].apply(discretise))
```

Now to apply to the data frames, including the combined dataset.

```
[13]: wine_red = add_alcohol_category(wine_red)
    wine_white = add_alcohol_category(wine_white)
    wine_all = add_alcohol_category(wine_all)
```

0.6.3 Task 1C: Discretise Alcohol Content

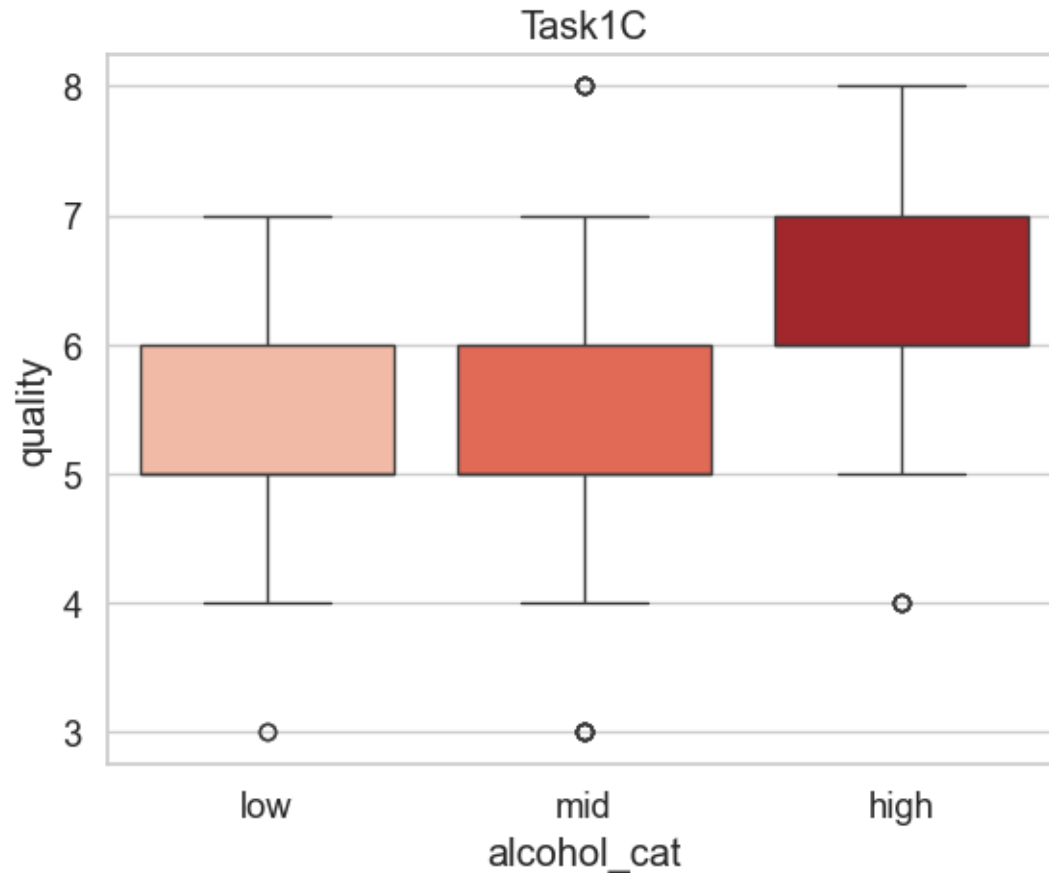
Now there is a discrete alcohol category, I can begin plotting.

Task1C states: “describe the distribution of wine quality as in (1.A), but separately for low-, mid-, and high-alcohol content.”.

As I am comparing the distribution of how alcohol affects wine quality, I should again draw box plots to compare and check peaks and mode with a violin plot.

I’ll start again using box plots, with red wine first:

```
[14]: sb.boxplot(
    x="alcohol_cat",
    hue="alcohol_cat",
    y="quality",
    data=wine_red,
    palette={
        "low": palette_red[2],
        "mid": palette_red[5],
        "high": palette_red[8]
    },
    order=["low", "mid", "high"],
    hue_order=["low", "mid", "high"],
)
plt.title("Task1C")
plt.savefig("../figures/task1c_box_red")
plt.show()
```



Box Plot Observations In the box plot above, for **red wines**:

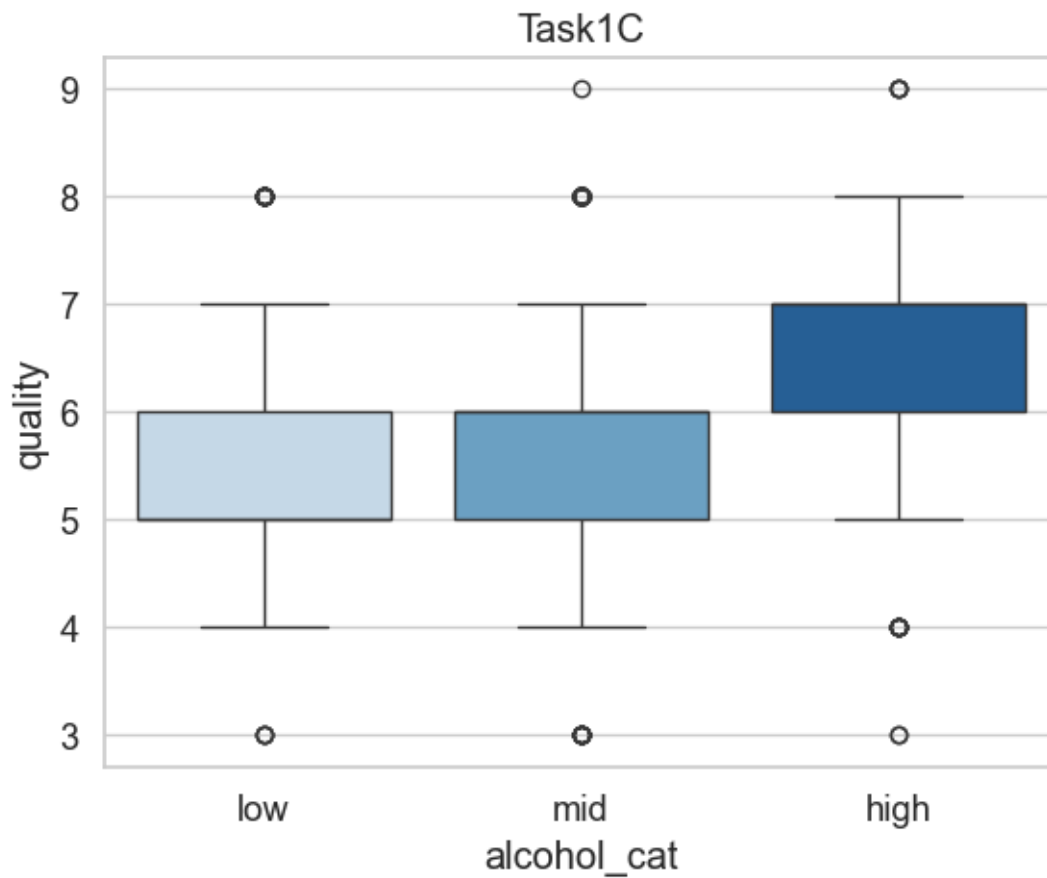
Statistic	low	mid	high
Median	5-6	5-6	6-7
IQR	5-6	5-6	6-7
Range	4-7	4-7	5-8
Outliers	3	3,8	4

The boxplot for **red wine** quality by alcohol shows: - Median quality increases for **high** alcohol wines. - **low** and **mid** categories have very similar spread (IQR), but **high** alcohol wines have larger spread. - Outliers at both small (3) and large (8) quality extremes are rare.

Box Plot Conclusion Higher alcohol content in wines once again indicates strong positive correlation with quality.

Now to plot the same data but this time for *white wine*:

```
[15]: sb.boxplot(
    x="alcohol_cat",
    hue="alcohol_cat",
    y="quality",
    data=wine_white,
    palette= {
        "low": palette_white[2],
        "mid": palette_white[5],
        "high": palette_white[8]
    },
    order=["low", "mid", "high"],
    hue_order=["low", "mid", "high"],
)
plt.title("Task1C")
plt.savefig("../figures/task1c_box_white")
plt.show()
```



Box Plot Observations Now, the box plot shows:

Statistic	low	mid	high
Median	5-6	5-6	6-7
IQR	5-6	5-6	6-7
Range	4-7	4-7	5-8
Outliers	3,8	3,8,9	3,4,9

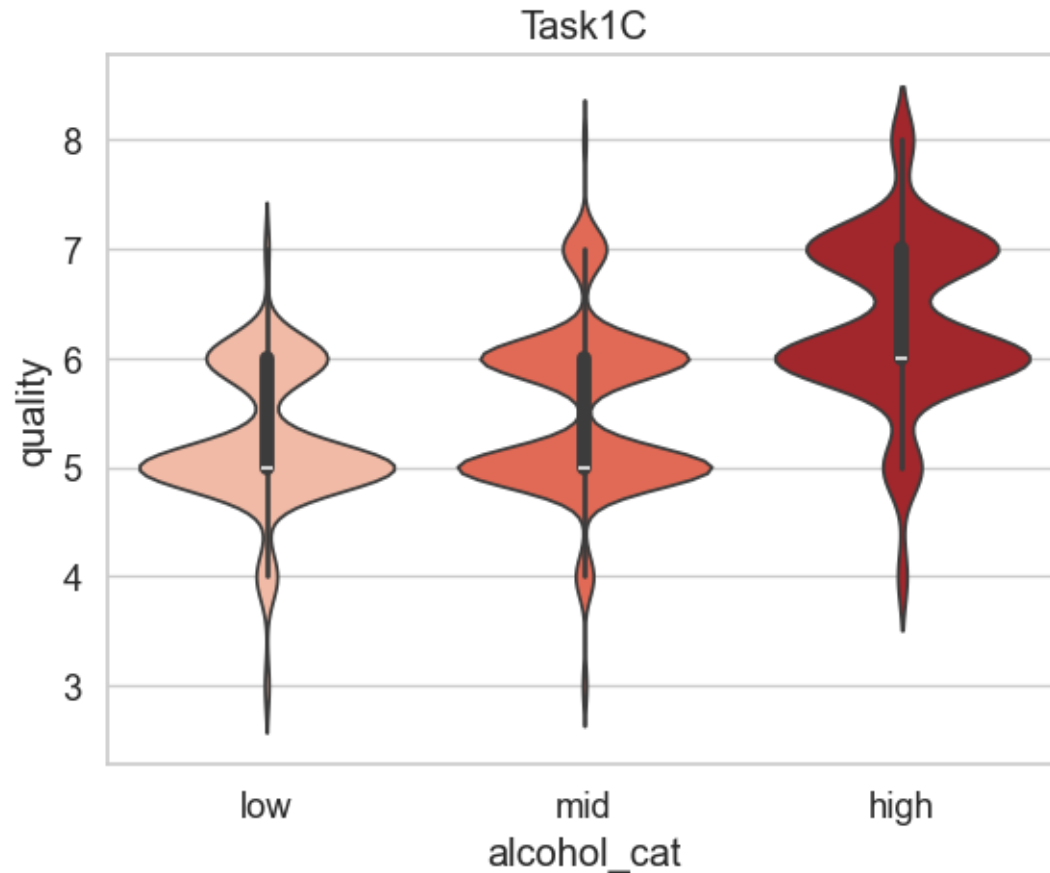
The boxplot for **white wine** quality is very similar to red wines': - Median rises from at **high** alcohol levels. - **high** alcohol levels have higher IQR (6-7) vs **low/mid** (5-6) - More outliers overall, especially higher values (9 in **high**, 8 and 9 in **mid**)

Box Plot Conclusion White wines also show a positive relationship between alcohol content and quality, maybe even more strongly than reds.

Now I'll plot both again, this time using violin plots. I'll start with red:

```
[16]: sb.violinplot(
    x="alcohol_cat",
    hue="alcohol_cat",
    y="quality",
    data=wine_red,
    palette= {
        "low": palette_red[2],
        "mid": palette_red[5],
        "high": palette_red[8]
    },
    order=["low", "mid", "high"],
    hue_order=["low", "mid", "high"],
)

plt.title("Task1C")
plt.savefig("../figures/task1c_violin_red")
plt.show()
```

Violin Plot Observations For **red wines**: - The density clusters confirm the box plot: peaks for **low** and **mid** are at quality 5–6, but for **high**, it's 6–7. - **high** category has visibly thicker (thus more) higher quality wines.

Violin Plot Conclusion The violin plot supports my analysis of the box plot — high alcohol levels in red wines correlate to higher quality ratings.

For the *white wines*:

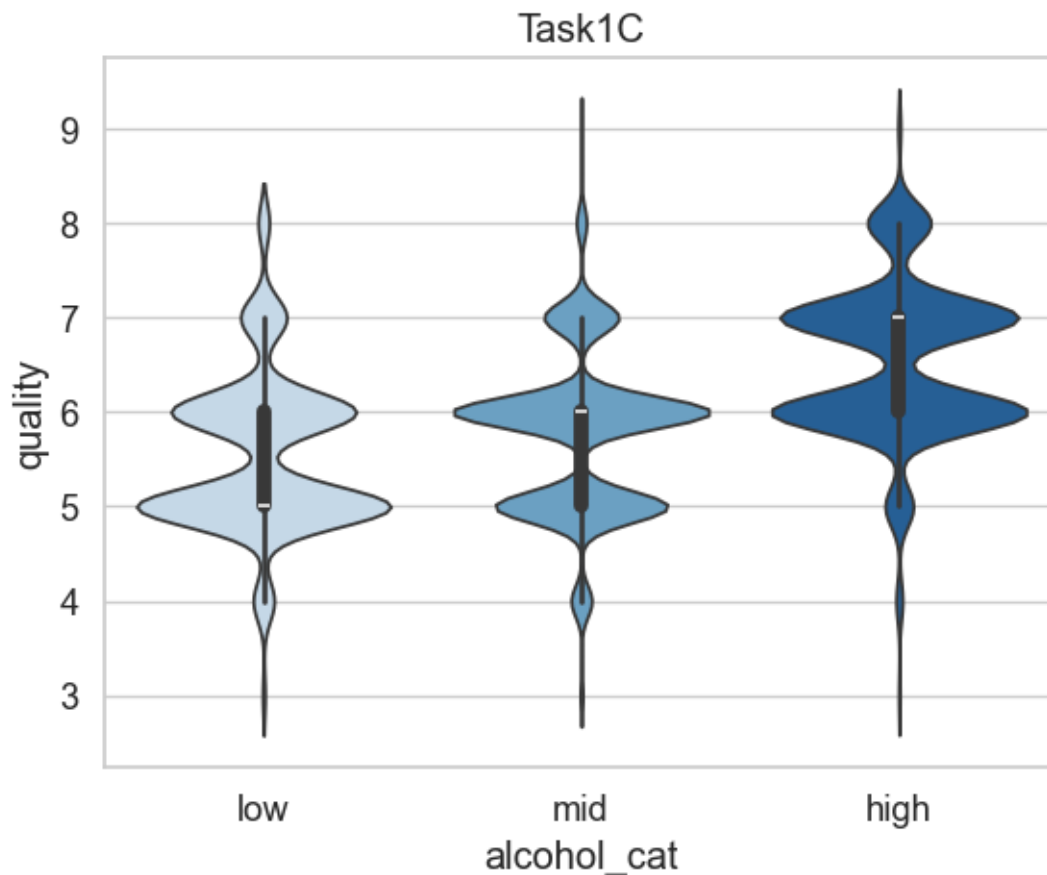
```
[17]: sb.violinplot(
      x="alcohol_cat",
      hue="alcohol_cat",
      y="quality",
      data=wine_white,
      palette= {
          "low": palette_white[2],
          "mid": palette_white[5],
          "high": palette_white[8]
      },
```

```

order=["low", "mid", "high"],
hue_order=["low", "mid", "high"],
)

plt.title("Task1C")
plt.savefig("../figures/task1c_violin_white")
plt.show()

```



Violin Plot Observations For **white wines**: - The distribution is broader than red wine, but the same trend is visible - positive correlation between alcohol content and quality. - low and mid cluster around quality 5–6 - high alcohol has a fatter top end, with more density at 7–8

Violin Plot Conclusion In white wines, high alcohol again correlates with higher quality.

Task 1C: Conclusion Both red and white wines show a clear positive association between alcohol content and quality. Wines in the high alcohol category consistently have higher medians, higher peaks, and more high end outliers.

For Later Modelling the consistent correlaton of alcohol content to quality suggests the alcohol

feature could be **useful in both regression and classification modelling**.

0.6.4 Task 1D

Now task 1C is complete, I'll move onto task 1D - *"plot the residual sugar variable and identify a suitable threshold to separate 'sweet' from 'dry' wines"* using an *"isSweet"* binary variable.

isSweet As seen in my earlier analysis, [Exploring the Dataset](#), the distribution of residual sugar is **skewed** for both red and white wines.

So, to examine it's affect on quality, I'll need to split the data evenly.

To do this, each record will be classified with an **isSweet** value of:

- 1, if residual sugar $\geq$ median,
- 0, if residual sugar $<$ median.

Using the **median** means there should be an almost even split between sweet and dry records, providing **balanced classes**.

```
[18]: def add_isSweet(df):  
        median = df["residual sugar"].median()  
        return df.assign(isSweet=(df["residual sugar"] >= median).astype(int))
```

And now to apply to all data frames:

```
[19]: wine_all = add_isSweet(wine_all)  
        wine_red = add_isSweet(wine_red)  
        wine_white = add_isSweet(wine_white)
```

To double check: the task (1D) states: *"find a threshold such that each class has approximately the same number of records"*.

The median should provide an even threshold, but to confirm:

```
[20]: isSweet_split = pd.DataFrame({  
        "All Wines": wine_all["isSweet"].value_counts(normalize=True),  
        "Red Wines": wine_red["isSweet"].value_counts(normalize=True),  
        "White Wines": wine_white["isSweet"].value_counts(normalize=True),  
    })  
isSweet_split.index = ["Dry (0)", "Sweet (1)"]  
isSweet_split
```

```
[20]:
```

	All Wines	Red Wines	White Wines
Dry (0)	0.503309	0.529706	0.501837
Sweet (1)	0.496691	0.470294	0.498163

And it works - each data frame has been split approximately evenly. That's **task 1D** complete!

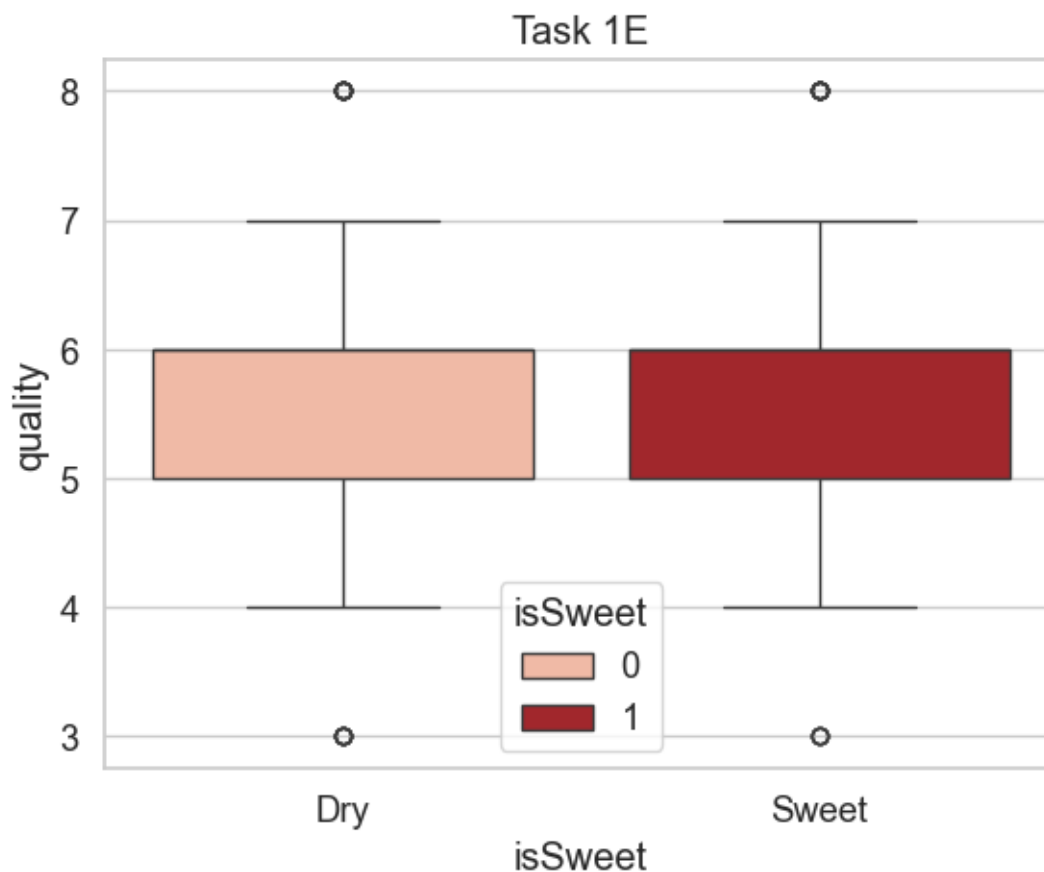
0.6.5 Task 1E: Plotting against isSweet

Now I'll begin **task 1E**: *"repeat the distribution analysis"* of quality, this time with **quality** vs **isSweet**.

Again, as I am comparing the *distribution* of how sweetness affects quality, I'll use box and violin plots.

For consistency, I'll start with box plots and red wine first:

```
[21]: sb.boxplot(  
    x="isSweet",  
    y="quality",  
    hue="isSweet",  
    data=wine_red,  
    palette={  
        0: palette_red[2],  
        1: palette_red[8]  
    },  
)  
plt.title("Task 1E")  
plt.xticks([0, 1], ["Dry", "Sweet"])  
plt.savefig("../figures/task1e_box_red")  
plt.show()
```



Box Plot Observations In the box plot above, for **red wines**:

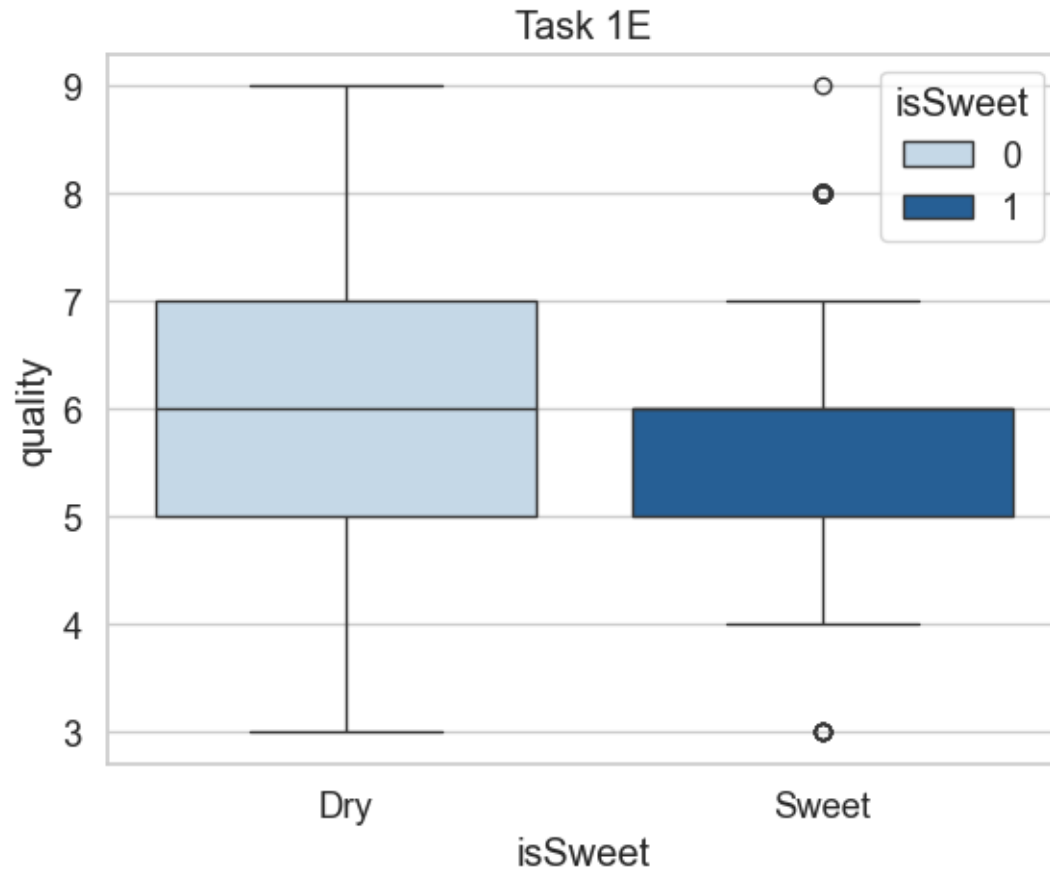
Statistic	dry	sweet
Median	5-6	5-6
IQR	5-6	5-6
Range	4-7	4-7
Outliers	3,8	3,8

The boxplot for **red wine** quality by `isSweet` shows: - Medians for dry and sweet wines are identical (5/6). - IQR is also the same (5 to 6). - Whiskers extend equally (4 to 7) - Some rare outliers at 3 and 8 for both datasets.

Box Plot Conclusion Sweet and dry red wines show no meaningful difference in quality. The distribution is visually identical, suggesting sweetness does not impact red wine quality in this dataset.

Now I'll plot the same data but this time for *white wine*:

```
[22]: sb.boxplot(  
    x="isSweet",  
    y="quality",  
    hue="isSweet",  
    data=wine_white,  
    palette={  
        0: palette_white[2],  
        1: palette_white[8]  
    },  
)  
plt.title("Task 1E")  
plt.xticks([0, 1], ["Dry", "Sweet"])  
plt.savefig("../figures/task1e_box_white")  
plt.show()
```



Box Plot Observations In the box plot above, for **white** wines:

Statistic	dry	sweet
Median	6	5-6
IQR	5-7	5-6
Range	3-9	4-7
Outliers	-	3,8,9

The boxplot for **white wine** quality by **isSweet** shows: - Dry wines have a higher median. - Dry wines also have a broader upper range, with qualities as high as 9. - Sweet wines are more concentrated between 5–6, with fewer high qualities.

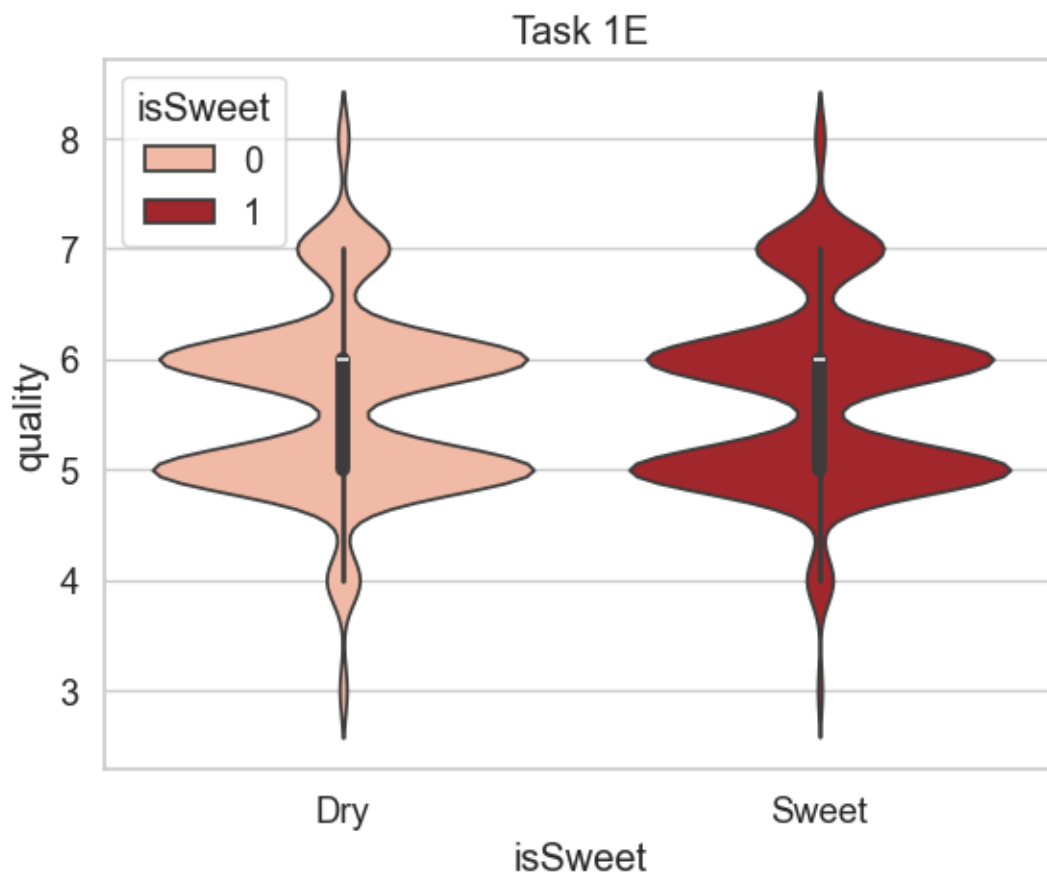
Box Plot Conclusions Dry white wines are generally higher quality than sweet ones, but for reds the box plots suggest sweetness does not affect quality.

Now to compare these findings against violin plots.

Red wine first:

```
[23]: sb.violinplot(
    x="isSweet",
    hue="isSweet",
    y="quality",
    data=wine_red,
    palette= {
        0: palette_red[2],
        1: palette_red[8]
    },
)

plt.title("Task 1E")
plt.xticks([0, 1], ["Dry", "Sweet"])
plt.savefig("../figures/task1e_violin_red")
plt.show()
```

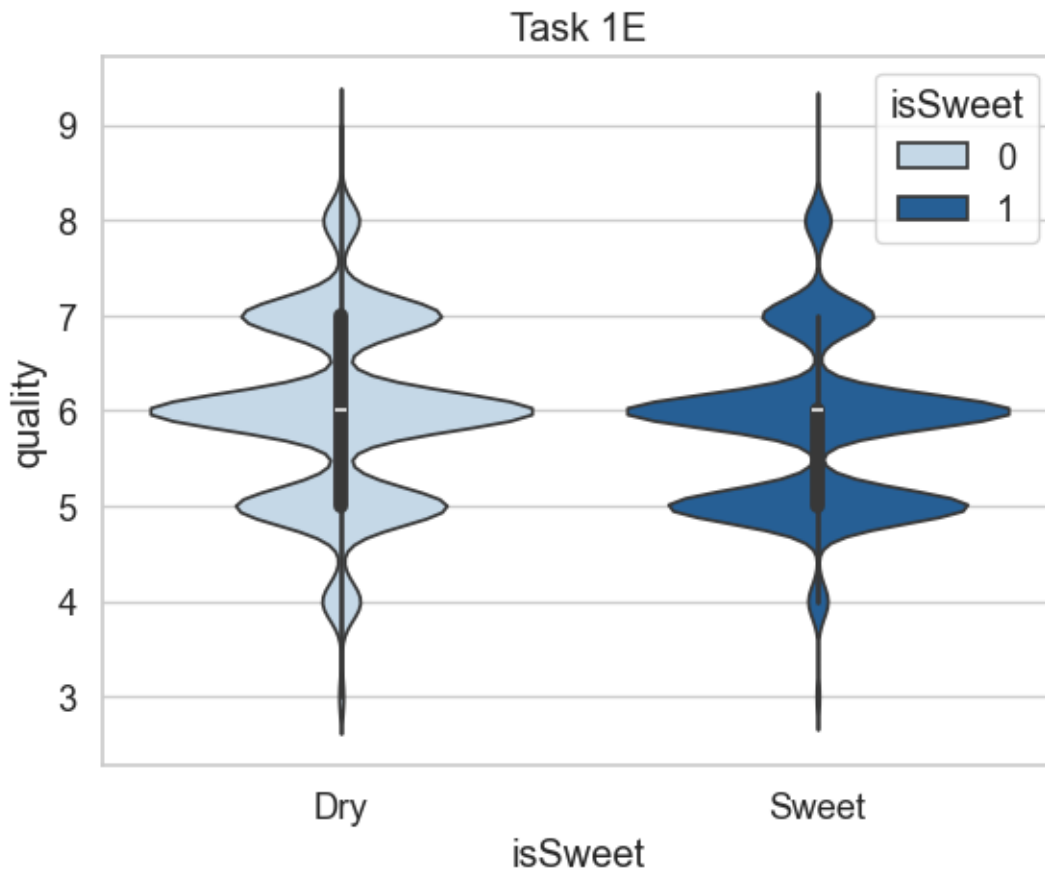


Violin Plot Observations In the plot above, for **red wines**: - Distribution shapes for dry and sweet reds are nearly identical. - Both groups show peaks at quality 5 and 6, with thinner tails at 4 and above 7. - The median and spread are once again visually identical.

Violin Plot Conclusion In red wines, sweetness has no clear effect on perceived quality. The violin plot confirms what the box plot showed — the distributions are hard to tell apart and almost the same.

```
[24]: sb.violinplot(
    x="isSweet",
    hue="isSweet",
    y="quality",
    data=wine_white,
    palette= {
        0: palette_white[2],
        1: palette_white[8]
    },
)

plt.title("Task 1E")
plt.xticks([0, 1], ["Dry", "Sweet"])
plt.savefig("../figures/task1e_violin_white")
plt.show()
```



Violin Plot Observations For **white wines**: - Dry white wines have a wider top half — more wines rated 7 and above. - Sweet white wines are concentrated around quality 5–6, with fewer high quality wines.

Violin Plot Conclusion In white wines, dry wines are more often rated higher than sweet ones. This violin plot makes the skew toward higher quality for dry whites and supports the previous box plot.

Task 1E Conclusion The generated violin plots and box plots supported each other, and suggested that white wine sweetness affects perceived quality, but red wine sweetness does not.

For Later Modelling: - The lack of correlation of sweetness to quality suggests the `isSweet` feature would be **unhelpful in later models for red wine quality** - The correlation between dry wines and higher quality could be a helpful predictor in **white wines quality** modelling.

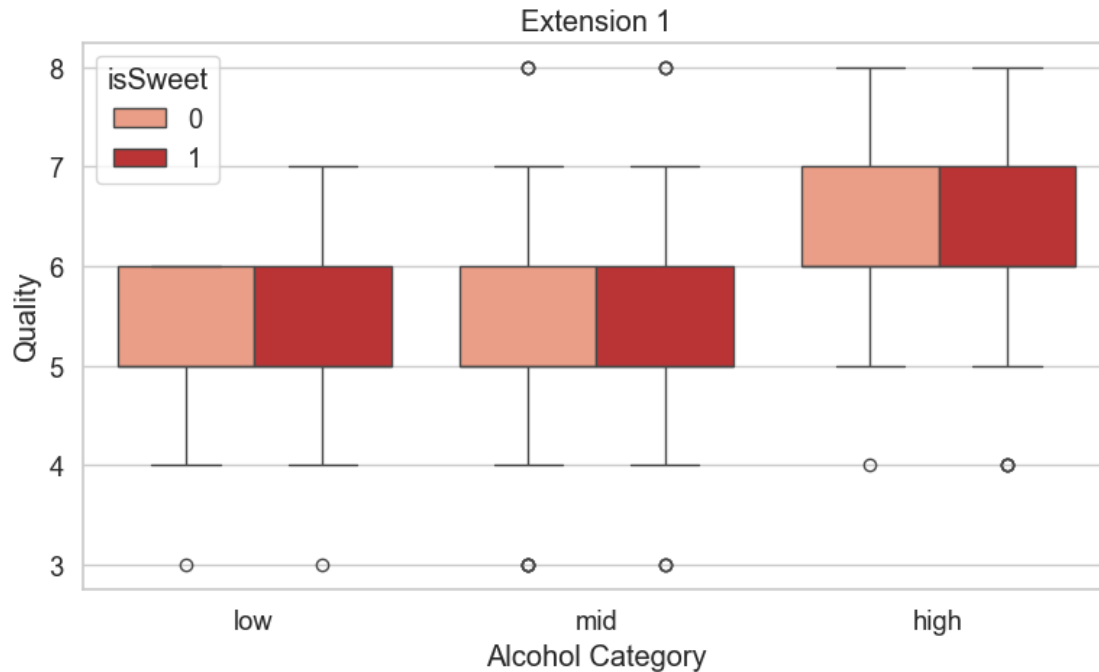
0.6.6 Extension 1: Plotting `isSweet` and `alcohol_cat`

Now I'll begin **the extension**: *"create suitable plots to characterise quality with respect to `isSweet` and `alcohol_cat`"*.

To do this, I'll start by plotting boxplots for the red wine dataset:

```
[25]: plt.figure(figsize=(8, 5))
      sb.boxplot(
          data=wine_red,
          x="alcohol_cat",
          y="quality",
          hue="isSweet",
          order=["low", "mid", "high"],
          palette={0: palette_red[3], 1: palette_red[7]}
      )

      plt.title("Extension 1")
      plt.xlabel("Alcohol Category")
      plt.ylabel("Quality")
      plt.legend(title="isSweet")
      plt.tight_layout()
      plt.savefig("../figures/exten1_red")
      plt.show()
```

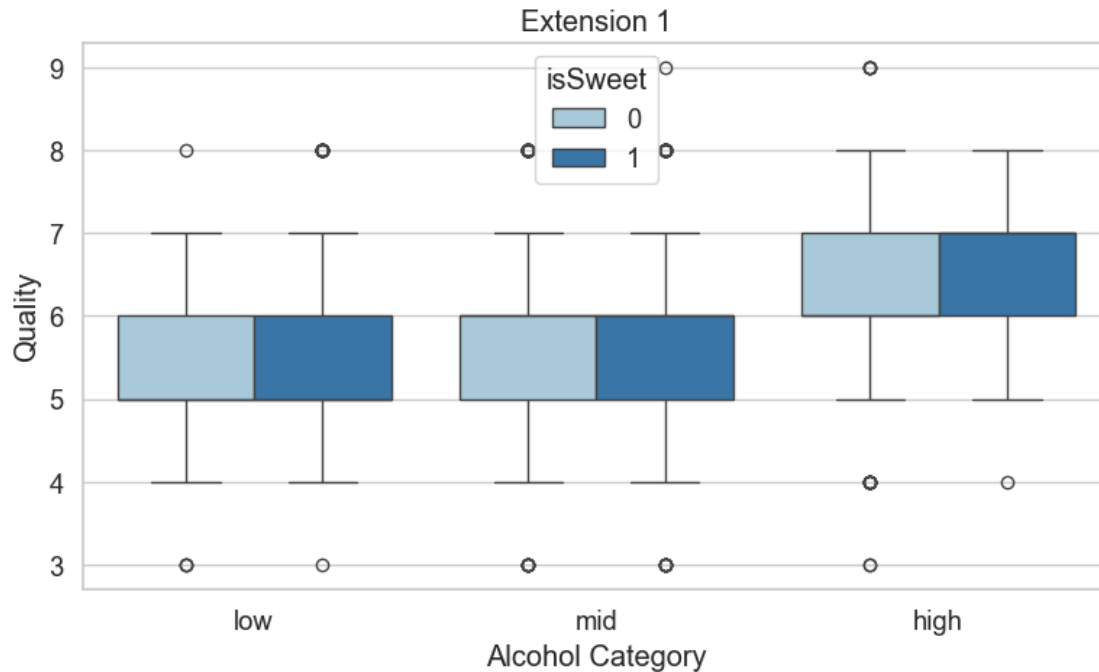


Observations The results show: - **Dry, high alcohol wines** have the highest quality on average.
- **Sweet, low alcohol wines** have the lowest quality on average, and show more variability.

These findings suggest that alcohol content has a strong positive relationship with quality, while sweetness and low alcohol may slightly reduce perceived quality in red wines.

Now, for white wines:

```
[26]: plt.figure(figsize=(8, 5))
      sb.boxplot(
          data=wine_white,
          x="alcohol_cat",
          y="quality",
          hue="isSweet",
          order=["low", "mid", "high"],
          palette={0: palette_white[3], 1: palette_white[7]}
      )
      plt.title("Extension 1")
      plt.xlabel("Alcohol Category")
      plt.ylabel("Quality")
      plt.legend(title="isSweet")
      plt.tight_layout()
      plt.savefig("../figures/exten1_white")
      plt.show()
```

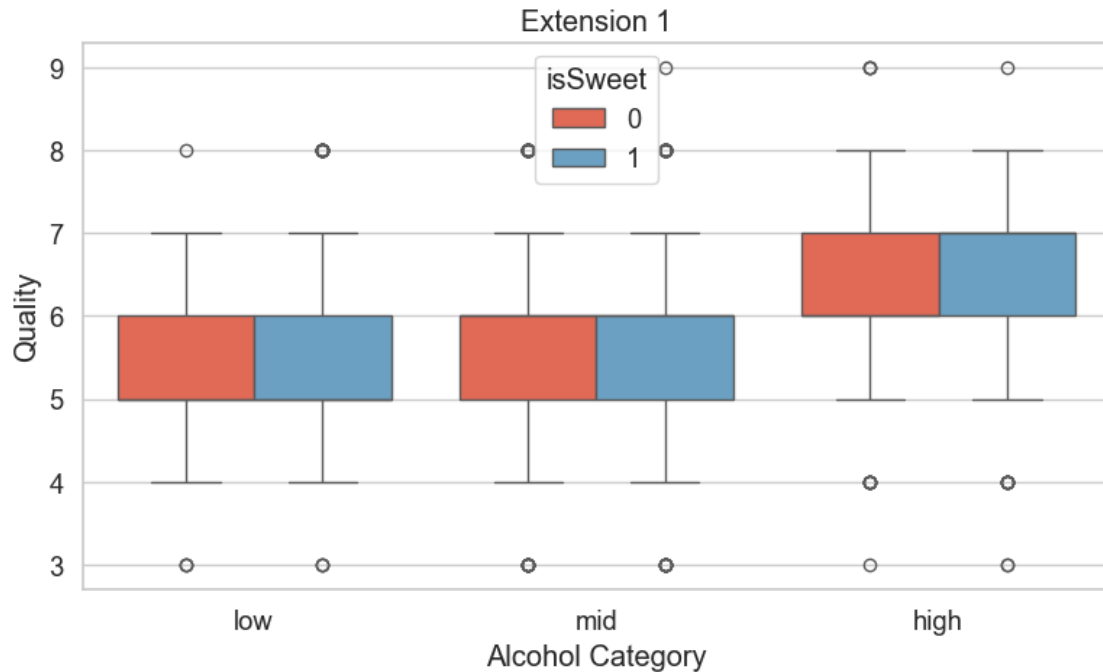


Observations This plot shows: - The distinction between dry and sweet is **less pronounced** in white wine than it was in red wine and suggests **alcohol content is a more important factor than sweetness** when it comes to predicting quality in white wines.

These results support earlier analysis and highlight how **combinations of features** can influence wine quality scores.

Now, to use both datasets and replot:

```
[27]: plt.figure(figsize=(8, 5))
      sb.boxplot(
          data=wine_all,
          x="alcohol_cat",
          y="quality",
          hue="isSweet",
          order=["low", "mid", "high"],
          palette={0: palette_red[5], 1: palette_white[5]}
      )
      plt.title("Extension 1")
      plt.xlabel("Alcohol Category")
      plt.ylabel("Quality")
      plt.legend(title="isSweet")
      plt.tight_layout()
      plt.savefig("../figures/exten1_both")
      plt.show()
```



Observations Plotting both datasets with respect to `isSweet` and `alcohol_cat` shows: - As expected, **higher alcohol content** is consistently correlated with **higher quality scores**, confirming its strong positive correlation across wine types. - Sweetness (`isSweet`) **does not show a dramatic effect** on quality when considered alongside alcohol category. In most cases, the difference between dry and sweet wines is minimal, especially within each alcohol group. - This suggests that when evaluating wine quality across both datasets, **alcohol is the better predictor** of quality.

0.7 Task 2: Variable Analysis

For **task 2**, the description reads: *“try and determine which subset of your variables can be most useful for learning”*.

To do this, it suggests: - Analysing each pair of variables - Analysing each variable and the quality outcome - Producing a visual representation in the form of a Correlation Matrix

0.7.1 Correlation Methods

Comparing the 3 suggested metrics, `pearson`, `kendall`, `spearman`:

Method	Description	Use when
Pearson	Measures linear correlations	Data is continuous and linear
Spearman	Measures monotonic correlation	Relationships may be non linear but ordered

Method	Description	Use when
Kendall	Measures concordance between ranked pairs	Small datasets

Justification for Spearman In this project, I chose to use **spearman** correlation because: - Wine quality is a **discrete variable** (measured from a scale of 1-10) - Many features (e.g. alcohol, acidity) may have **non linear** but **monotonic** relationships with quality - **spearman** handles these cases better than **pearson**, which assumes all relationships are linear - The datasets are large, especially the white wine dataset, so **kendall** may not be as useful

Thus **spearman** is the most appropriate choice for the correlation analysis.

0.7.2 Predictions:

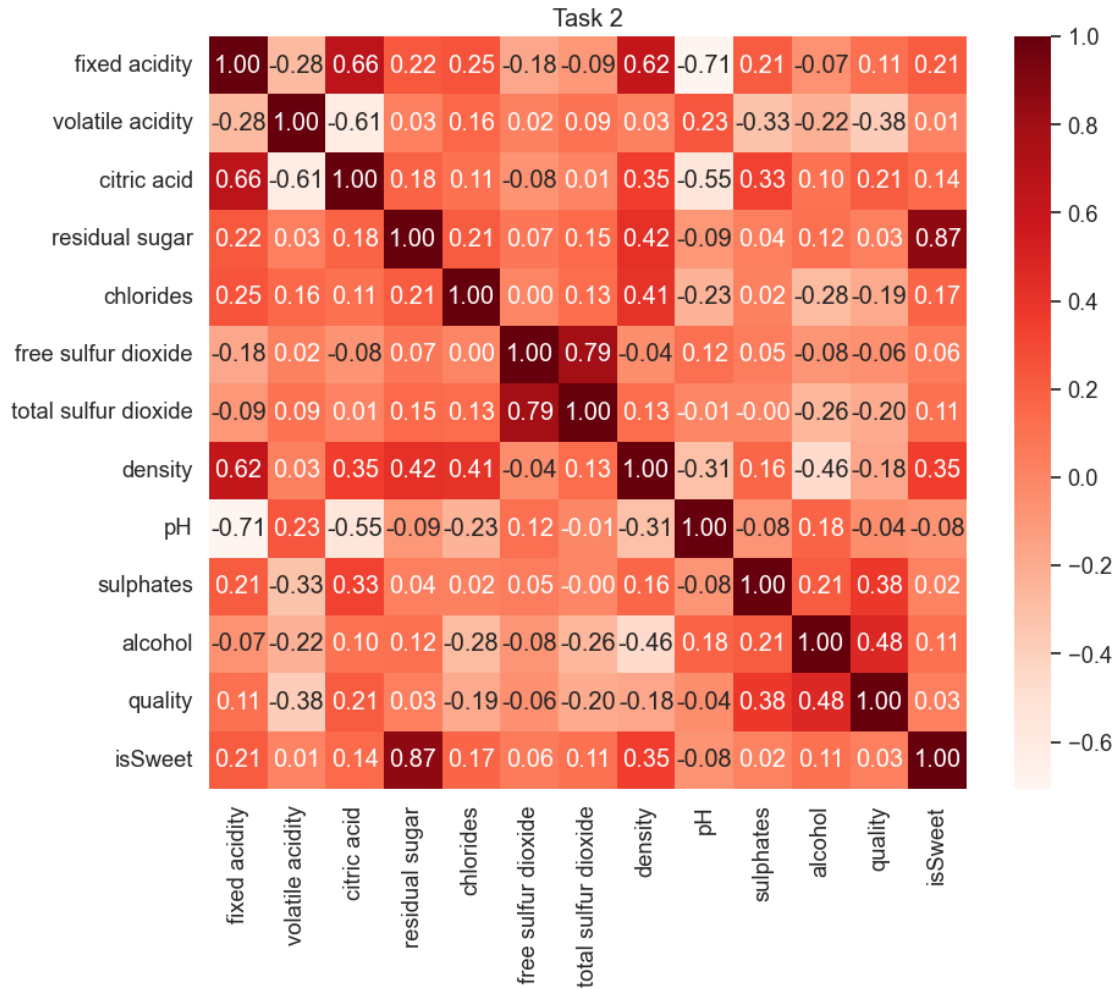
Based on the previous analysis, I predict: - Alcohol content will be a strong indicator of quality - Volatile acidity content will be a negative indicator of quality - Sweetness will be a strong indicator of quality for **white wines**

Below, I'll define the correlation matrix first for each dataset:

```
[28]: spearman_cm_red = wine_red.corr(numeric_only=True, method="spearman")
      spearman_cm_white = wine_white.corr(numeric_only=True, method="spearman")
```

Then, I'll plot the matrix for red wines:

```
[29]: plt.figure(figsize=(10, 8))
      sb.heatmap(spearman_cm_red, annot=True, cmap="Reds", fmt=".2f")
      plt.title("Task 2")
      plt.savefig("../figures/task2_red.png")
      plt.show()
```



Red Wine Variable Correlations For red wines, the most prominent relationships to quality are:

Top Features	Correlation with <code>quality</code>
<code>alcohol</code>	0.48
<code>sulphates</code>	0.38
<code>volatile acidity</code>	-0.38
<code>citric acid</code>	0.21
<code>total sulfur dioxide</code>	-0.20

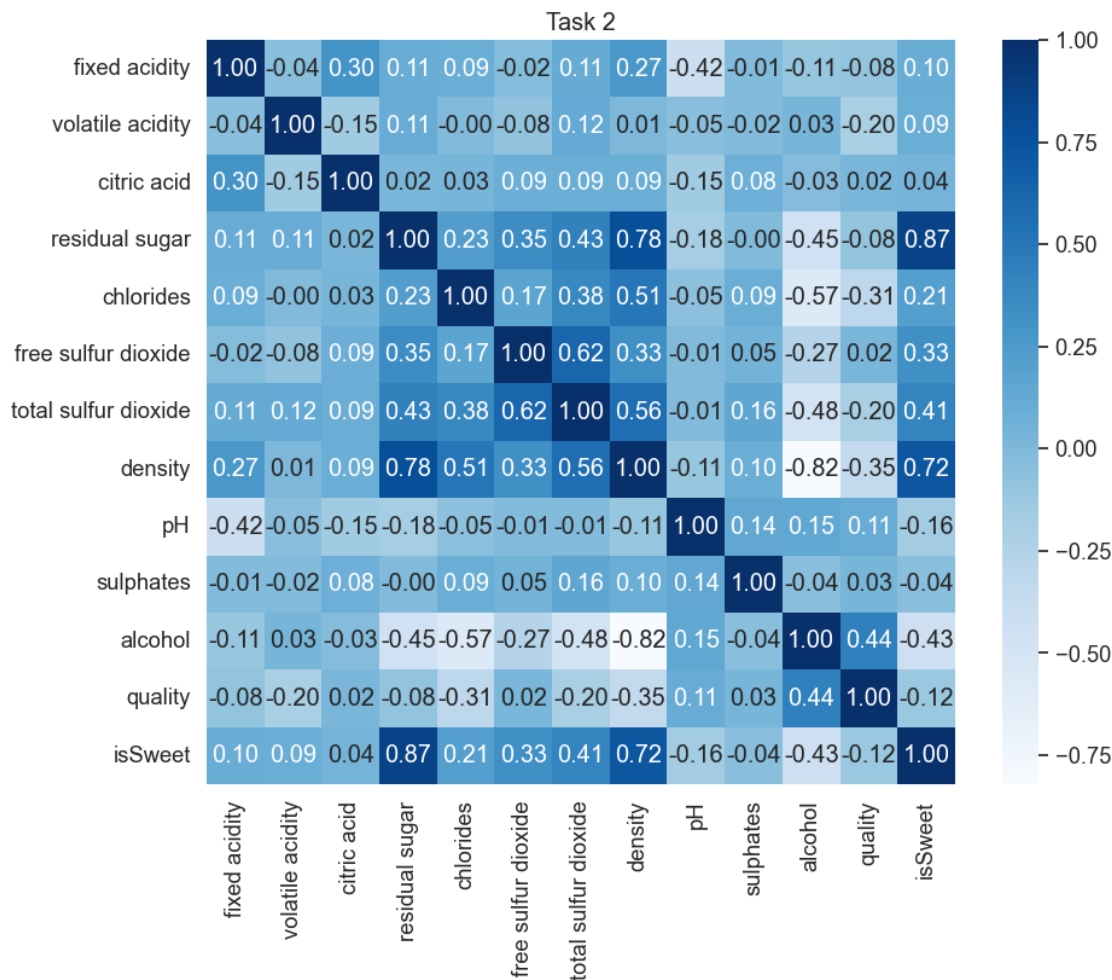
Other Features:

- `alcohol` and `sulphates` are the most positively associated features with higher quality.
- `volatile acidity` shows a strong negative correlation — higher levels appear to reduce quality.

- isSweet, pH, residual sugar, and free sulfur dioxide, show very weak or negligible correlation and may be less useful.

Now I'll do the same for white wines:

```
[30]: plt.figure(figsize=(10, 8))
sb.heatmap(spearman_cm_white, annot=True, cmap="Blues", fmt=".2f")
plt.title("Task 2")
plt.savefig("../figures/task2_white.png")
plt.show()
```



White Wine Variable Correlations For white wine, the strongest relationships to quality are:

Top Features	Correlation with quality
alcohol	0.44
density	-0.35
chlorides	-0.31

Top Features	Correlation with quality
total sulfur dioxide	-0.20
volatile acidity	-0.20
isSweet	-0.12
residual sugar	-0.08

Other Features:

- `alcohol` is the most positively associated with higher quality.
- `density`, `chlorides`, `total sulfur dioxide` and `volatile acidity` show fair negative correlation — higher levels appear to reduce quality.
- `citric acid`, `free sulfur dioxide`, `sulfates` and `fixed acidity`, show very weak or negligible correlation for white wines and may be less useful for predicting quality.
- `isSweet` is a weak indicator of poor quality.

0.7.3 Overall

Based on the correlation analysis for both red and white wines:

The **most helpful features** for predicting wine quality across both datasets are: - `alcohol` (strong positive correlation) - `sulphates` (only in red wine) - `density` and `chlorides` (negative correlation in white wine) - `volatile acidity` and `total sulfur dioxide` (negative correlations)

Features like `citric acid`, `residual sugar`, `free sulfur dioxide` (`isSweet` for reds) and `pH` show **weak or no correlation** and should be excluded from modelling.

Keeping features very highly correlated with each other, like `density/isSweet` and `residual sugar`, or `free sulfur dioxide` and `total sulfur dioxide`, may be redundant.

0.7.4 Final Feature Set for *Red Wine*

Feature	Correlation with quality	Why?
<code>alcohol</code>	+0.48	Strongest positive predictor
<code>sulphates</code>	+0.38	Good positive predictor
<code>volatile acidity</code>	-0.38	Strong negative predictor
<code>citric acid</code>	+0.21	Mild positive
<code>density</code>	+0.35	Could be useful, especially with <code>isSweet</code>
<code>total sulfur dioxide</code>	-0.20	
<code>isSweet</code>	+0.11	Useful for binary classification

Dropping: - `residual sugar` - `free sulfur dioxide` - `pH`

Final Feature Set for *White Wine*

Feature	Correlation with quality	Why?
alcohol	+0.44	Strongest predictor again
density	−0.35	Strong inverse relationship
chlorides	−0.31	Strongest mineral predictor
total sulfur dioxide	−0.20	Moderate negative
volatile acidity	−0.20	Same as red wine
isSweet	−0.12	Same as red wine

Dropping: - residual sugar - citric acid - sulphates - fixed acidity - pH

0.8 Task 3: Modelling

Now I've explored the data and analysed relationships, I can begin modelling.

Approach Task 3 is to “*experiment with one or more machine learning approaches that predict wine quality from the datasets variables*”.

I'll explore two different approaches; **binary classification** and **regression**.

To do this, I'll split **task 3** into two different sections:

- (A) **Binary classification**: Try and predict quality scores as **classes** based on a given threshold.
- (B) **Regression**: Try and predict the **exact** quality score.

0.8.1 Models

Initially, I'll use the **Random Forest** model for both binary classification and regression. This is the most popular model on Kaggle for this dataset and well documented. Since this is new territory, well-documented notebooks serve as great tutorials and comparing results will help me identify erroneous behaviour.

I will briefly explore other models alongside **Random Forest**, but will use RF as the primary baseline for comparisons.

Now, before I begin training, I'll need to set up my data.

I'll start by dropping the features I don't want to use in training for the **red wine dataset** (including the “guessed” feature `quality`, styling feature `colour` and duplicate `alcohol_cat`).

```
[31]: X_red = wine_red.drop(columns=["quality", "type", "residual sugar", "isSweet",
    ↪ "free sulfur dioxide", "alcohol_cat", "pH"])
y_red = wine_red["quality"]
```

Next, I'll import `test_train_split` from `sklearn` and use it to divide my dataset into training and test data, using an 80/20 split:

```
[32]: from sklearn.model_selection import train_test_split
```

```
X_red_train, X_red_test, y_red_train, y_red_test = train_test_split(
    X_red, y_red, test_size=0.2, random_state=1
)
```

0.8.2 Task 3A: Binary Classification

Now I can begin binary classification.

First, I'll need to import and define my chosen classifier (`RandomForestClassifier`) fixing the randomness to make results reproducible.

```
[33]: from sklearn.ensemble import RandomForestClassifier
      rfc = RandomForestClassifier(random_state=42)
```

Next, I'll define the thresholds to test:

```
[34]: thresholds = [5, 6, 7]
```

Then, I'll import the relevant libraries for training.

For now, I won't use the recommended metrics in order to keep things simple.

Instead, I will use the `accuracy_score` metric, (which calculates simply `number of correct predictions / number of true values`) to determine which threshold is best.

To store each thresholds' metric, I'll create an empty dict.

```
[35]: from sklearn.pipeline import Pipeline
      from sklearn.metrics import accuracy_score

      accuracy_scores = {}
```

Now I'll use the **red wine data** and iterate through the thresholds.

Each loop, I'll train my classifier 1000 times, storing the accuracy in my defined list above.

```
[36]: for threshold in thresholds:
      print("-" * 42)
      print(f"Train with threshold: `high quality > {threshold}`")

      y_red_bin = (y_red > threshold).astype(int)
      scores = []

      # TBR: Change back to 1000
      for _ in range(10):
          X_train, X_test, y_train, y_test = train_test_split(
              X_red, y_red_bin, test_size=0.2, stratify=y_red_bin
          )

          pipeline = Pipeline([
              ("classifier", rfc)
          ])

```

```

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
scores.append(accuracy_score(y_test, y_pred))

accuracy_scores[threshold] = scores

```

Train with threshold: `high quality > 5`

Train with threshold: `high quality > 6`

Train with threshold: `high quality > 7`

And check the mean of the results for each threshold:

```

[37]: print("| Threshold | Accuracy (%) |")
print("|-----|-----|")
for threshold in thresholds:
    scores = accuracy_scores[threshold]
    mean_accuracy = sum(scores) / len(scores)
    print(f"| 'high' > {threshold:<2} | {mean_accuracy * 100:>10.2f} |")

```

Threshold	Accuracy (%)
-----	-----
'high' > 5	80.84
'high' > 6	91.28
'high' > 7	98.84

These results show that **accuracy increases as the threshold increases**. However, this does **not** necessarily mean the model is improving.

As the threshold rises, fewer wines are labelled as “high quality”, creating a **more imbalanced dataset**. When almost all wines belong to one class (e.g. most red wines have quality < 7 thus belong to low), the model can achieve high accuracy by simply predicting the **low** class.

This suggests that the classifier might be **biased**, especially at **threshold > 7**, where the accuracy is almost **99%**.

To verify this, I’ll use these findings to plot accuracy distributions using a **histogram** and inspect class balances:

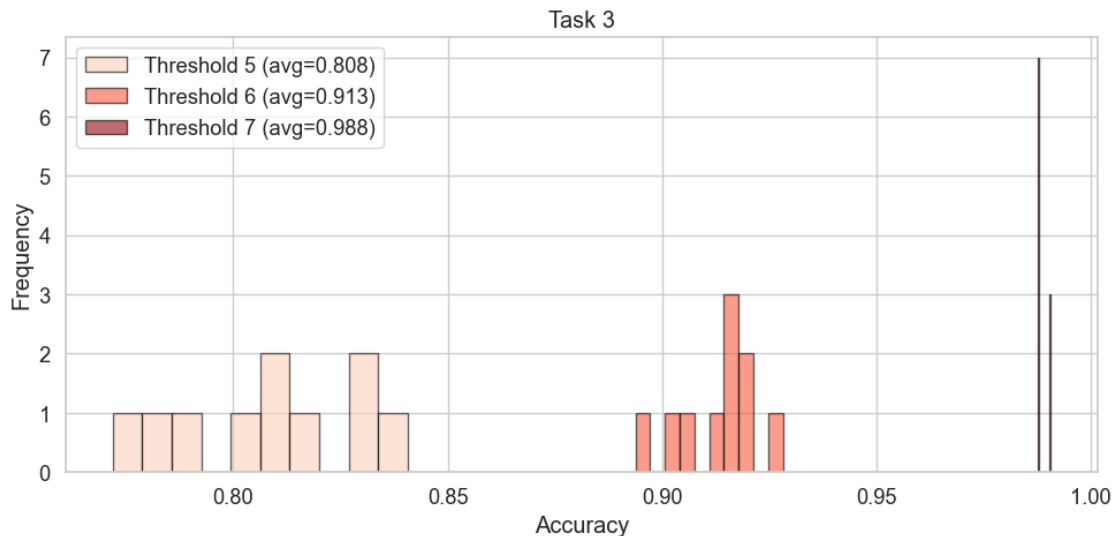
```

[38]: plt.figure(figsize=(10, 5))

for threshold in thresholds:
    scores = accuracy_scores[threshold]
    plt.hist(scores, bins=10, alpha=0.6, label=f"Threshold {threshold} (avg={np.
    ↪mean(scores):.3f})",
            color=palette_red[1 if threshold == 5 else 5 if threshold == 6
    ↪else 9], edgecolor="black")

```

```
plt.xlabel("Accuracy")
plt.ylabel("Frequency")
plt.title("Task 3")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("../figures/task3_histogram_red.png")
plt.show()
```



Here, I can see for:

- Threshold 5: Accuracy ranges mostly from approximately 0.78–0.86.
- Threshold 6: Smaller range, from approximately 0.89–0.93.
- Threshold 7: Extreme — many scores very near to 1 (likely due to major class imbalance).

I can improve the model to avoid class imbalances.

Right now, the model behaves slightly differently depending on which subset it trains on, with accuracy ranging from high 70's to high 90's.

To avoid this, I'll implement **cross validation**. I'll start by importing `cross_val_score` from `sklearn`.

```
[39]: from sklearn.model_selection import cross_val_score
```

And I'll make an empty dict to again store the scores:

```
[40]: cv_scores_by_threshold = {}
```

And finally apply 10 fold cross validation to each threshold:

```
[41]: for threshold in thresholds:
        y_red_bin = (y_red > threshold).astype(int)
        scores = cross_val_score(rfc, X_red, y_red_bin, cv=10)
        cv_scores_by_threshold[threshold] = scores
```

Now, to check for any difference, I'll plot again as a histogram:

```
[42]: plt.figure(figsize=(10, 5))

for threshold in thresholds:
    scores = cv_scores_by_threshold[threshold]
    plt.hist(scores, bins=10, alpha=0.6, label=f"Threshold {threshold} (avg={np.
    ↪mean(scores):.3f})",
            color=palette_red[1 if threshold == 5 else 5 if threshold == 6
    ↪else 9], edgecolor="black")

plt.xlabel("Accuracy")
plt.ylabel("Frequency")
plt.title("Task 3 CV")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("../figures/task3_histogram_cv_red.png")
plt.show()
```



Now I can see:

- Threshold 5: Wide spread again, much larger range (approx. 0.57-0.82)
- Threshold 6: Smaller range than threshold 5; average around 0.9.

- Threshold 7: Nearly perfect accuracy again.

There is still too much class imbalance in threshold 7. To view further scores and identify whether threshold 7 is truly learning, I'll use the `sklearn classification_report`.

First, I'll again train the model, this time **only** on threshold 7:

```
[43]: y_red_bin_7 = (y_red > 7).astype(int)
X_train, X_test, y_train, y_test7 = train_test_split(X_red, y_red_bin_7,
    ↪test_size=0.2, stratify=y_red_bin_7, random_state=42)

rfc.fit(X_train, y_train)
y_pred7 = rfc.predict(X_test)
```

Now I'll have a look at how many times it predicts class low vs high:

```
[44]: from sklearn.metrics import classification_report

print(classification_report(y_test7, y_pred7, target_names=["Low", "High"],
    ↪zero_division=0))
```

	precision	recall	f1-score	support
Low	0.99	1.00	0.99	316
High	0.00	0.00	0.00	4
accuracy			0.99	320
macro avg	0.49	0.50	0.50	320
weighted avg	0.98	0.99	0.98	320

Looking at the report for threshold 7, it's clear that the model strongly favours predicting low for the quality of the wines.

I can tell this as: - Recall for **high** is 0.20: this means it has only identified 20% of actual high quality wines. - F1-score for **high** is 0.33: suggesting a very imbalanced performance.

Now, I'll compare this to the other thresholds, starting with 5, where the class distribution is more balanced and the model may be learning and detecting both classes.

```
[45]: y_red_bin_5 = (y_red > 5).astype(int)
X_train, X_test, y_train, y_test5 = train_test_split(X_red, y_red_bin_5,
    ↪test_size=0.2, stratify=y_red_bin_5, random_state=42)

rfc.fit(X_train, y_train)
y_pred5 = rfc.predict(X_test)
```

And then I'll look at the report:

```
[46]: print(classification_report(y_test5, y_pred5, target_names=["Low", "High"],
    ↪zero_division=0))
```

	precision	recall	f1-score	support
Low	0.77	0.78	0.78	149
High	0.81	0.80	0.80	171
accuracy			0.79	320
macro avg	0.79	0.79	0.79	320
weighted avg	0.79	0.79	0.79	320

At threshold 5, the model achieves a much more realistic performance. It's lower than at threshold 7, but this is a good thing: at 7, the model achieved misleadingly high accuracy (0.99) by ignoring the **high** class.

Here, the distribution is more balanced, allowing the model to generalise better across **both** classes.

This makes threshold 5 a better choice for binary classification than 7, but it had lower accuracy than both other thresholds, so I'll review threshold 6's performance too:

```
[47]: y_red_bin_6 = (y_red > 6).astype(int)
X_train, X_test, y_train, y_test6 = train_test_split(X_red, y_red_bin_6,
    ↪test_size=0.2, stratify=y_red_bin_6, random_state=42)

rfc.fit(X_train, y_train)
y_pred6 = rfc.predict(X_test)
```

Now to view 6's classification report:

```
[48]: print(classification_report(y_test6, y_pred6, target_names=["Low", "High"],
    ↪zero_division=0))
```

	precision	recall	f1-score	support
Low	0.94	0.99	0.97	277
High	0.93	0.60	0.73	43
accuracy			0.94	320
macro avg	0.94	0.80	0.85	320
weighted avg	0.94	0.94	0.94	320

Threshold 6 shows:

- F1-score for **high** is 0.72

Compared to threshold 5 (balanced but lower precision and recall), threshold 6 provides a better trade off: it preserves the model's ability to detect **high** quality wines **without** sacrificing as much performance on the **low** class.

Unlike threshold 7, where the model ignored wines classed as **high** and produced misleading accuracy, threshold 6 shows the most balanced and effective performance overall with a strong F1-score for both classes.

This suggests threshold 6 is the best threshold for binary classification in this task, and it's what I'll use going forward.

Now I've seen how Random Forest performs with different thresholds and selected the most suitable, I'll define some more models to try perform the same binary classification.

For new models, I'm going to scale my data. To do so, I'll first redefine my test and training data as so:

```
[49]: X_red_train, X_red_test, y_red_train, y_red_test = train_test_split(X_red,
    ↪ y_red_bin_6, test_size=0.2, stratify=y_red_bin_6, random_state=42)
```

Then I'll scale based on that:

```
[50]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_red_scaled = scaler.fit_transform(X_train)
X_test_red_scaled = scaler.transform(X_test)
```

Now I'll define new models:

Logical Regression model:

```
[51]: from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(max_iter=10000, random_state=42,
    ↪ class_weight="balanced",)
```

Support Vector Match model:

```
[52]: from sklearn.svm import SVC
svc = SVC(kernel='rbf', probability=True, random_state=42,
    ↪ class_weight="balanced")
```

And Gradient Boosting Classification model:

```
[53]: from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(random_state=42)
```

To compare results, I'll define another dict to hold scores:

```
[54]: evaluation_results = {}
```

I'll also define a helper function to evaluate each model going forward. This will store the models scores and display them with a classification report and the average accuracy after applying 10 fold cross validation.

```
[55]: def evaluate_model(model, X_train, X_test, y_train, y_test, name):
    scores = cross_val_score(model, X_train, y_train, cv=10, scoring='f1')
    print(f"{name} - 10 fold CV F1-score: {scores.mean():.3f} +- {scores.std():.
    ↪ 3f}")

    model.fit(X_train, y_train)
```



```

y_pred = model.predict(X_test)

report = classification_report(y_test, y_pred, target_names=["Low", "High"],
                               output_dict=True)
evaluation_results[name] = {
    "F1 High": report["High"]["f1-score"],
    "F1 Low": report["Low"]["f1-score"],
    "Accuracy": report["accuracy"],
    "CV F1": scores.mean()
}

print(classification_report(y_test, y_pred, target_names=["Low", "High"]))

```

Now to compare how these perform compared to my Random Forest model.

I'll start with Logistic Regression:

```
[56]: evaluate_model(logreg, X_train_red_scaled, X_test_red_scaled, y_red_train,
                    y_red_test, "LogReg")
```

LogReg - 10 fold CV F1-score: 0.478 +- 0.050

	precision	recall	f1-score	support
Low	0.97	0.81	0.88	277
High	0.40	0.81	0.54	43
accuracy			0.81	320
macro avg	0.68	0.81	0.71	320
weighted avg	0.89	0.81	0.84	320

Logistic Regression – Threshold 6 Evaluation Despite overall high recall scores, the precision and F1-score for high quality wines is low. So, the logistic regression model underperforms Random Forest in this task.

Next, I'll try SVC:

```
[57]: evaluate_model(svc, X_train_red_scaled, X_test_red_scaled, y_red_train,
                    y_red_test, "SVC")
```

SVC - 10 fold CV F1-score: 0.505 +- 0.059

	precision	recall	f1-score	support
Low	0.96	0.83	0.89	277
High	0.42	0.77	0.54	43
accuracy			0.82	320
macro avg	0.69	0.80	0.72	320
weighted avg	0.89	0.82	0.84	320

SVC – Threshold 6 SVC slightly outperforms logistic regression on the **high** class, with improved balance, but is still not higher than Random Forest.

I'll move onto Gradient Boosting:

```
[58]: evaluate_model(gb, X_train_red_scaled, X_test_red_scaled, y_red_train,
    ↪y_red_test, "GB")
```

GB - 10 fold CV F1-score: 0.505 +- 0.103

	precision	recall	f1-score	support
Low	0.93	0.99	0.96	277
High	0.88	0.51	0.65	43
accuracy			0.93	320
macro avg	0.90	0.75	0.80	320
weighted avg	0.92	0.93	0.92	320

Gradient Boosting – Threshold 6 GB has good overall performance, especially in recall, but still has inconsistent precision across classes and therefore *still* underperforms Random Forest.

Lastly, to really compare to Random Forest, I'll fine tune some parameters so the model doesn't use an **imbalanced class**.

First, I'll redefine my model, then compare that to the others:

```
[59]: rfc = RandomForestClassifier(n_estimators=200, max_depth=10,
    ↪class_weight="balanced", random_state=42)
evaluate_model(rfc, X_train_red_scaled, X_test_red_scaled, y_red_train,
    ↪y_red_test, "RF 2")
```

RF 2 - 10 fold CV F1-score: 0.589 +- 0.126

	precision	recall	f1-score	support
Low	0.95	0.97	0.96	277
High	0.78	0.67	0.72	43
accuracy			0.93	320
macro avg	0.87	0.82	0.84	320
weighted avg	0.93	0.93	0.93	320

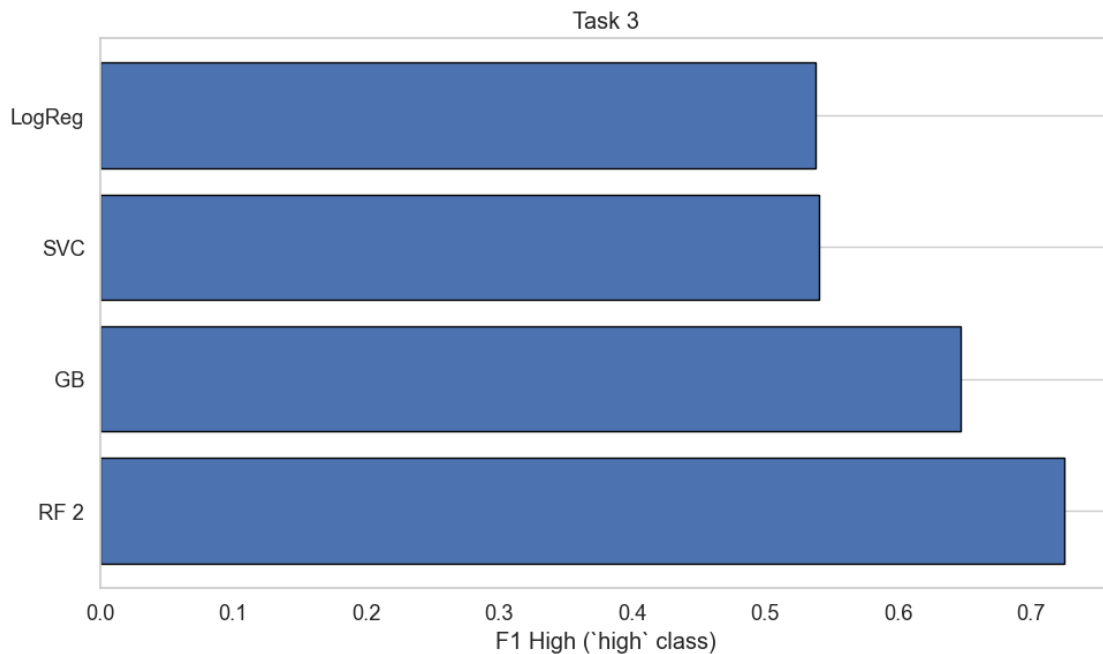
Random Forest 2 – Threshold 6 This model achieves the best overall balance of **precision (0.79)** and **recall (0.70)** for the high class.

It outperforms all others in both test performance and in cross-validation scores. The final, tuned Random Forest is clearly the most effective model for binary classification.

And, to visualise the results, I'll plot a bar chart of each model against it's F1-Score for the **high** class:

```
[60]: labels = list(evaluation_results.keys())
      f1_scores = [evaluation_results[model]["F1 High"] for model in labels]

      plt.figure(figsize=(10, 6))
      plt.barh(labels, f1_scores, edgecolor="black")
      plt.xlabel("F1 High (`high` class)")
      plt.title("Task 3")
      plt.gca().invert_yaxis()
      plt.grid(axis='x')
      plt.tight_layout()
      plt.show()
```



Summary - Binary Classification Using the plot, it's clear that the tuned Random Forest algorithm performs the best in binary classification (at least when comparing the F1 scores for each **high** class together), and threshold 6 is the most balanced threshold to use.

Now, I'll move onto exploring regression.

0.8.3 Task 3B: Regression

Now I've finished binary classification, I'll try modelling using quality as a **continuous variable** by using **regression**, again only with the red wine dataset for now.

To begin, I'll again split my data into testing and training sets:

```
[61]: X_red_train, X_red_test, y_red_train, y_red_test = train_test_split(X_red,
      ↪ y_red, test_size=0.2, stratify=y_red_bin_6, random_state=42)
```

Then rescale:

```
[62]: X_train_red_scaled = scaler.fit_transform(X_train)
      X_test_red_scaled = scaler.transform(X_test)
```

Now, to define and fit my model. First I'll try again with a Random Forest model - this time, Random Forest Regressor. I won't fine tune any parameters just yet, either.

```
[63]: from sklearn.ensemble import RandomForestRegressor
      rfr = RandomForestRegressor(random_state=42)
      rfr.fit(X_train_red_scaled, y_red_train)
```

```
[63]: RandomForestRegressor(random_state=42)
```

To measure success for my regressor, I'll compare the following metrics: - Mean squared error (MSE) - Root MSE (RMSE) - R2 score

And import them from sklearn:

```
[64]: from sklearn.metrics import mean_squared_error, r2_score
```

Now I'll have a quick look at how a baseline Random Forest performs:

```
[65]: y_pred_rfr = rfr.predict(X_test_red_scaled)
      mse = mean_squared_error(y_red_test, y_pred_rfr)
      rmse = mse ** 0.5
      r2 = r2_score(y_red_test, y_pred_rfr)
      print(f"Random Forest Regressor\nMSE: {mse:.3f} | RMSE: {rmse:.3f} | R2: {r2:.3f}")
```

```
Random Forest Regressor
MSE: 0.359 | RMSE: 0.599 | R2: 0.511
```

Random Forest Regressor The model has a RMSE of almost 0.6, which is quite high, but a rounded R2 of 0.51 - this suggests the model explains just over half of the variance in quality scores and is a fine baseline.

Now I'll try with another model - SVC:

```
[66]: from sklearn.svm import SVR
      svr = SVR(kernel='rbf')
      svr.fit(X_train_red_scaled, y_red_train)
```

```
[66]: SVR()
```

And to look at the results:

```
[67]: y_pred_svr = svr.predict(X_test_red_scaled)
      mse = mean_squared_error(y_red_test, y_pred_svr)
      rmse = mse ** 0.5
      r2 = r2_score(y_red_test, y_pred_svr)
```

```
print(f"SVR\nMSE: {mse:.3f} | RMSE: {rmse:.3f} | R2: {r2:.3f}")
```

SVR

MSE: 0.408 | RMSE: 0.639 | R2: 0.444

SVR SVR underperforms compared to the **Random Forest Regressor**: it recieved a lower R2 and also got a higher RMSE.

This suggests that SVR may be less effective at modelling and the Random Forest model remains the stronger choice for this regression task so far.

Now another model, I'll try GBR:

```
[68]: from sklearn.ensemble import GradientBoostingRegressor
      gbr = GradientBoostingRegressor(random_state=42)
      gbr.fit(X_train_red_scaled, y_red_train)
```

```
[68]: GradientBoostingRegressor(random_state=42)
```

And to look at results:

```
[69]: y_pred_gbr = gbr.predict(X_test_red_scaled)
      mse = mean_squared_error(y_red_test, y_pred_gbr)
      rmse = mse ** 0.5
      r2 = r2_score(y_red_test, y_pred_gbr)
      print(f"GBR\nMSE: {mse:.3f} | RMSE: {rmse:.3f} | R2: {r2:.3f}")
```

GBR

MSE: 0.421 | RMSE: 0.649 | R2: 0.427

GBR Compared to the **Random Forest Regressor**, the GBR again underperforms slightly in both prediction accuracy (RMSE) and explained variance (R2).

As I did with Binary Classification, I'll try fine tuning the best forming model, Random Forest Regressor, to see if I can improve results.

This time, I'll use `RandomizedSearchCV` to find the best parameters for tuning.

```
[70]: from sklearn.model_selection import RandomizedSearchCV

      parameters = {
          'n_estimators': [100, 200, 300],
          'max_depth': [10, 20, 30, None],
          'min_samples_split': [2, 5, 5],
          'min_samples_leaf': [1, 2, 4],
          'max_features': ['sqrt', 'log2'],
      }

      random_search = RandomizedSearchCV(
          estimator=rfr,
          param_distributions=parameters,
```

```

n_iter=30,
scoring='neg_root_mean_squared_error',
cv=10,
verbose=1,
n_jobs=-1,
random_state=42
)

random_search.fit(X_train_red_scaled, y_red_train)
best_rfr = random_search.best_estimator_
print("Best parameters:", random_search.best_params_)

```

Fitting 10 folds for each of 30 candidates, totalling 300 fits

```
Best parameters: {'n_estimators': 200, 'min_samples_split': 2,
'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': None}
```

Now, using the parameters provided, I'll reevaluate my Random Forest model:

```
[71]: y_pred_best_rfr = best_rfr.predict(X_test_red_scaled)
mse = mean_squared_error(y_red_test, y_pred_best_rfr)
rmse = mse ** 0.5
r2 = r2_score(y_red_test, y_pred_best_rfr)

print(f"Param Tuned Random Forest\nRMSE: {rmse:.3f} | R²: {r2:.3f}")

```

Param Tuned Random Forest

RMSE: 0.588 | R²: 0.530

Random Forest Regressor 2 After tuning with 10-fold cross-validation and RandomizedSearchCV, the fine tuned Random Forest regressor improved the R2 score over the baseline Random Forest but dropped in the RMSE.

Summary: Task 3 During task 3, I trained different models for both binary classification and regression, and briefly compared results. For binary classification, I tested and chose a threshold for the classification based on how the model performed with each and plotted F1-scores on a histogram to compare results. For regression, I explored parameter tuning with my Random Forest model using GridSearchCV and compared how initial and new results differed.

0.9 Task 4: Model Evaluation

0.9.1 Model Evaluation

Task 4 was to evaluate the model using *k fold cross-validation* - I have already used **10 fold cross-validation** in some instances above and have briefly compared results using a handful of the tasks' suggested error metrics.

For proper evaluation, I will apply 10 fold CV consistently on both models and regenerate new metrics. The two different approaches I tried, as seen in Task 3, were: 1. **Binary classification**
2. **Regression**

For Task 4, I'll also add the remainder of the suggested metrics to evaluate models and generate plots: this is in order to really learn about the performance of my models.

0.9.2 Binary Classification Review

To evaluate performance for Binary Classification, I previously plotted all models against F1-scores for the **high** class. Now, I'll use two other metrics (**ROC curve and AUC score**) on my best performing classifier, Random Forest.

To start, I'll import the metrics from `sklearn`:

```
[72]: from sklearn.metrics import roc_curve, auc
```

Resplit into testing and training data:

```
[73]: y_red_bin_6 = (y_red > 6).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X_red, y_red_bin_6,
    ↪test_size=0.2, random_state=42, stratify=y_red_bin_6)
```

Again train my model:

```
[74]: rfc = RandomForestClassifier(n_estimators=200, max_depth=10,
    ↪class_weight="balanced", random_state=42)
rfc.fit(X_train, y_train)
```

```
[74]: RandomForestClassifier(class_weight='balanced', max_depth=10, n_estimators=200,
    random_state=42)
```

Predict class and generate ROC and AUC scores alongside previously seen F1-score:

```
[75]: y_prob = rfc.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
f1_scores = cross_val_score(rfc, X_red, y_red_bin_6, cv=10, scoring="f1")
```

And I'll print these all together:

```
[76]: print(f"F1-score: {f1_scores.mean():.3f} +- {f1_scores.std():.3f}")
print(f"AUC score: {roc_auc:.3f}")
```

F1-score: 0.459 +- 0.078

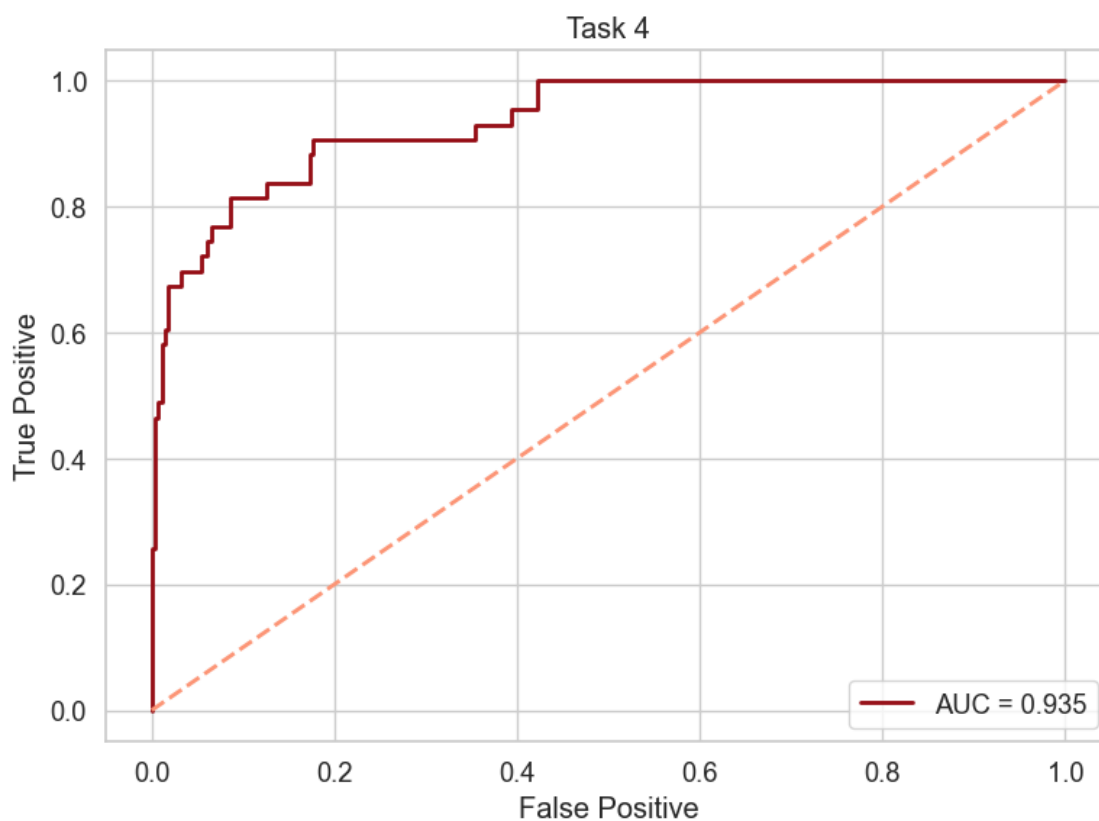
AUC score: 0.935

Observations On the single train/test split previously used in Task 3, the model achieved an F1-score of around 0.7. However, when evaluated using *10 fold cross-validation*, the average F1-score dropped to a rounded 0.46. This is not ideal, and suggests that the model is struggling to predict high quality wines. This is probably due to there being very few high quality red wines.

Though, the AUC is high: approximately 0.94. This suggests that while the model is good at ranking instances (as shown by AUC), its thresholded classification performance varies more depending on the data split — likely due to class imbalance and threshold sensitivity.

Finally, I'll plot the ROC curve:

```
[77]: plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color=palette_red[9], lw=2, label=f"AUC = {roc_auc:.3f}")
plt.plot([0, 1], [0, 1], color=palette_red[3], lw=2, linestyle="--")
plt.xlabel("False Positive")
plt.ylabel("True Positive")
plt.title("Task 4")
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig("../figures/task4_roc_red.png")
plt.show()
```



Observations The ROC curve above shows the performance of the Random Forest classifier for separating between high and low quality (red) wines, where “high quality” is defined as wines with a quality score greater than 6.

- The **curve rises sharply** toward the top-left corner, indicating a high true positive rate with

a relatively low false positive rate.

- The **AUC** is about **0.9**, as seen above, confirming that the model has **high discriminative ability**.
- The dashed diagonal line represents random guessing; since the ROC curve lies well above this, the classifier outperforms arbitrary guessing.

0.9.3 Regression Review

To evaluate my best regression model (again, Random Forest) I'll use mainly the same metrics I used in Task 3 alongside...

I also didn't previously generate any plots, so I will do so here.

To begin, I'll split my data into testing and training:

```
[78]: X_train, X_test, y_train, y_test = train_test_split(X_red, y_red, test_size=0.
      ↪2, random_state=42)
```

Retrain the Random Forest Regressor model:

```
[79]: rfr = RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42)
      rfr.fit(X_train, y_train)
```

```
[79]: RandomForestRegressor(max_depth=10, n_estimators=200, random_state=42)
```

Import to define CV:

```
[80]: from sklearn.model_selection import KFold
      from sklearn.model_selection import cross_val_predict
```

Let the model predict:

```
[81]: y_pred = rfr.predict(X_test)
```

Predict and calculate metrics using **10 fold cv**:

```
[82]: cv = KFold(n_splits=10, shuffle=True, random_state=42)
      nmse_scores = cross_val_score(rfr, X_red, y_red,
      ↪scoring="neg_mean_squared_error", cv=cv)
      mae_scores = -cross_val_score(rfr, X_red, y_red, cv=cv,
      ↪scoring="neg_mean_absolute_error")
      mse_scores = -nmse_scores
      rmse_scores = np.sqrt(mse_scores)
      r2_scores = cross_val_score(rfr, X_red, y_red, cv=cv, scoring="r2")
```

Now, I'll look at how the model performed:

```
[83]: print(f"MSE:  {mse_scores.mean():.3f} +- {mse_scores.std():.3f}")
      print(f"RMSE: {rmse_scores.mean():.3f} +- {rmse_scores.std():.3f}")
      print(f"MAE:  {mae_scores.mean():.3f} +- {mae_scores.std():.3f}")
      print(f"R2:   {r2_scores.mean():.3f} +- {r2_scores.std():.3f}")
```

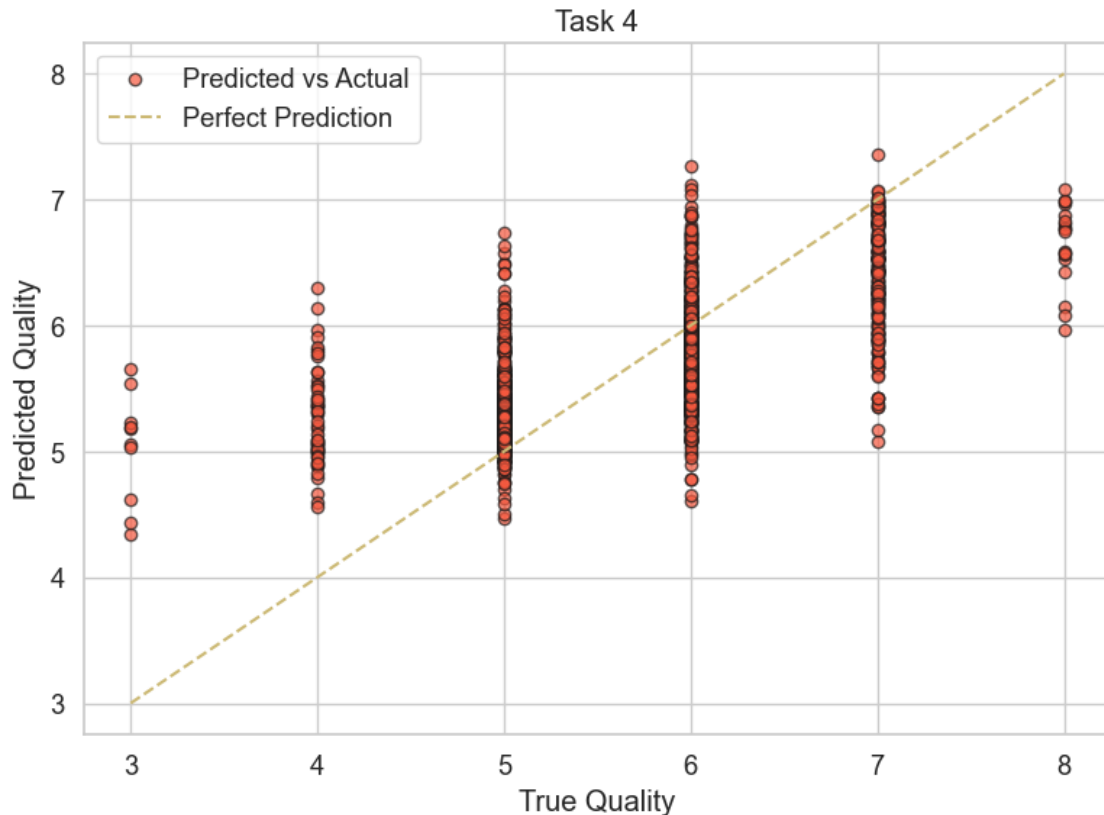
MSE: 0.339 +- 0.038
RMSE: 0.581 +- 0.032
MAE: 0.433 +- 0.026
R2: 0.476 +- 0.070

Observations These results indicate that the model predicts wine quality with error of **about half a point** on the 0–10 quality scale. The low standard deviations across folds suggest stable generalisation. The **R2 score** means that the model explains nearly **48% of the variance** in wine quality — a solid result given the subjectivity of wine quality (differs on consumer basis). Overall, this regression model performs reliably and generalises well across data splits.

Now, to plot for the regression model each predicted value against the true value of quality:

```
[84]: y_pred_cv = cross_val_predict(rfr, X_red, y_red, cv=cv)

plt.figure(figsize=(8, 6))
plt.scatter(y_red, y_pred_cv, color=palette_red[5], edgecolors="k",
            ↪label="Predicted vs Actual", alpha=0.7)
plt.plot([y_red.min(), y_red.max()], [y_red.min(), y_red.max()], "y--",
            ↪label="Perfect Prediction")
plt.xlabel("True Quality")
plt.ylabel("Predicted Quality")
plt.title("Task 4")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("../figures/task4_regression_red.png")
plt.show()
```



Observations In the plot, it's clear most predictions vary, somewhat close to the diagonal line of "*perfect prediction*". This confirms that the model has general accuracy, and is typically within a 1 point range for most wines qualities. However, it does underperform in the case of outliers (3 and 8). This is acceptable given the discrete nature of wine quality scores (integers from 3 to 8) and that quality varies based on customer preferences

Overall, the 10 fold cross-validated regression model has good performance and generalisation, but is not excellent.

0.9.4 White Wines

Although my modelling exploration has focused heavily on the red wine dataset, the **white wine dataset may be more useful**. In my exploratory analysis, it was seen to have a larger range and has over 3x as many points as the red wine dataset. This may help my class imbalance problem and improve my models performances'.

Thus, I'll apply the same modelling techniques to the white wine dataset and compare:

0.9.5 White Wine Binary Classification

I'll start with binary classification, using the best threshold - 6.

First, I'll drop the defined features in Task 2:

```
[85]: X_white = wine_white.drop(columns=["quality", "type", "residual sugar",
    ↪ "alcohol_cat", "citric acid", "sulphates", "fixed acidity", "pH"])
y_white = wine_white["quality"]
```

Split into test and train data using **threshold 6**:

```
[86]: y_white_bin_6 = (y_white > 6).astype(int)
X_train, X_test, y_train, y_test = train_test_split(X_white, y_white_bin_6,
    ↪ test_size=0.2, random_state=42, stratify=y_white_bin_6)
```

Train the Random Forest Classifier on this data:

```
[87]: rfc = RandomForestClassifier(n_estimators=200, max_depth=10,
    ↪ class_weight="balanced", random_state=42)
rfc.fit(X_train, y_train)
y_prob = rfc.predict_proba(X_test)[: , 1]
```

Calculate metrics:

```
[88]: fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
f1_scores = cross_val_score(rfc, X_white, y_white_bin_6, cv=10, scoring="f1")
```

And display them:

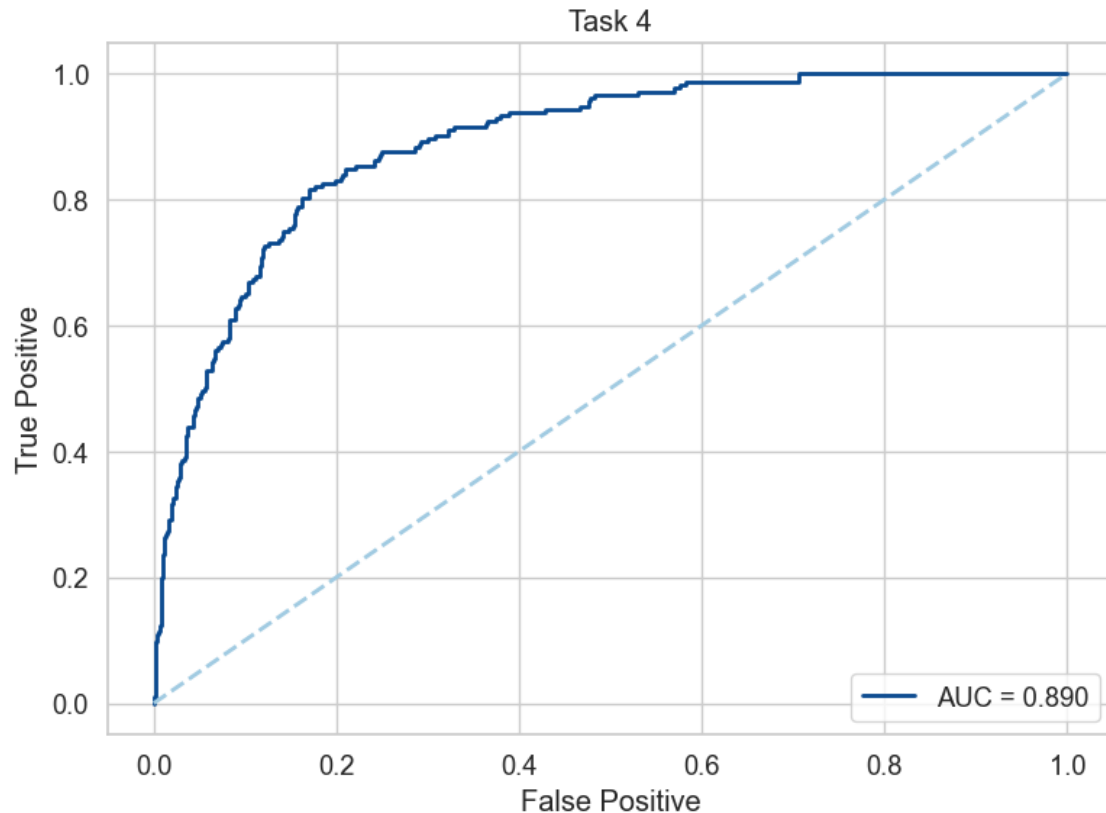
```
[89]: print(f"F1-score (white): {f1_scores.mean():.3f} +- {f1_scores.std():.3f}")
print(f"AUC score (white): {roc_auc:.3f}")
```

F1-score (white): 0.539 +- 0.051

AUC score (white): 0.890

Observations While the red wine data achieved a higher AUC, indicating better overall ability to rank high vs low qualities, the white wine model produced a **higher F1-score**. This suggests it handled class imbalance better and made more balanced predictions. In short, the white wine classifier was **more effective at identifying high quality wines using the same threshold and process**, even if slightly less confident in ranking them overall.

```
[90]: plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color=palette_white[9], lw=2, label=f"AUC = {roc_auc:.3f}")
plt.plot([0, 1], [0, 1], color=palette_white[3], lw=2, linestyle="--")
plt.xlabel("False Positive")
plt.ylabel("True Positive")
plt.title("Task 4")
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig("../figures/task4_roc_white.png")
plt.show()
```



Observations Although the white wine classifier had a **higher F1-score**, its ROC curve suggests it's a little less confident in ranking predictions compared to the red wine model.

0.9.6 White Wine Regression

Now I'll do the same for the white wine dataset and regression modelling.

Split data:

```
[91]: X_train, X_test, y_train, y_test = train_test_split(X_white, y_white,
    ↪ test_size=0.2, random_state=42)
```

Define model and CV and fit:

```
[92]: rfr = RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42)
cv = KFold(n_splits=10, shuffle=True, random_state=42)
rfr.fit(X_train, y_train)
```

```
[92]: RandomForestRegressor(max_depth=10, n_estimators=200, random_state=42)
```

Calculate error metrics:

```
[93]: mse_scores = -cross_val_score(rfr, X_white, y_white, cv=cv,
    ↪scoring="neg_mean_squared_error")
rmse_scores = np.sqrt(mse_scores)
mae_scores = -cross_val_score(rfr, X_white, y_white, cv=cv,
    ↪scoring="neg_mean_absolute_error")
r2_scores = cross_val_score(rfr, X_white, y_white, cv=cv, scoring="r2")
```

And display scores:

```
[94]: print(f"MSE:  {mse_scores.mean():.3f} +- {mse_scores.std():.3f}")
print(f"RMSE: {rmse_scores.mean():.3f} +- {rmse_scores.std():.3f}")
print(f"MAE:  {mae_scores.mean():.3f} +- {mae_scores.std():.3f}")
print(f"R2:   {r2_scores.mean():.3f} +- {r2_scores.std():.3f}")
```

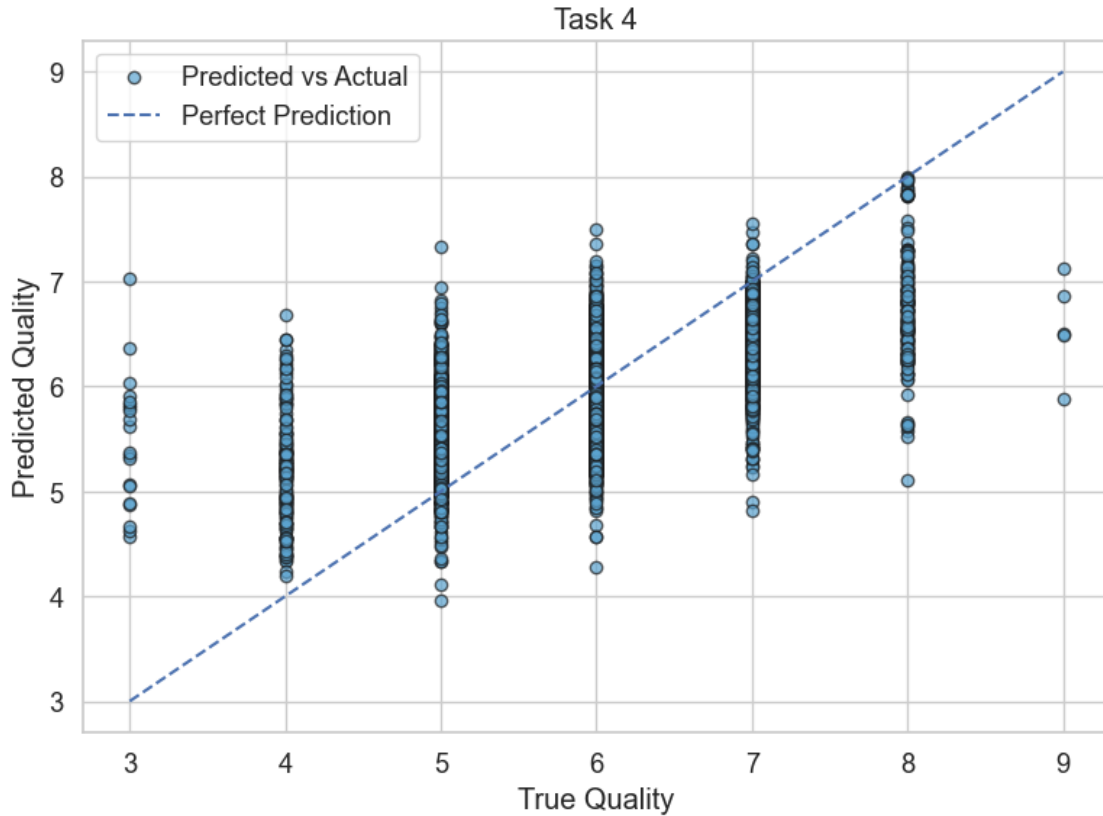
```
MSE:  0.438 +- 0.030
RMSE: 0.662 +- 0.023
MAE:  0.511 +- 0.013
R2:   0.439 +- 0.040
```

Compared to the red wine model, on white wine dataset the regressor shows **slightly worse predictive accuracy**, with lower error metrics across the board and a higher R2 score. This suggests that **wine quality is slightly easier to model in the red wine dataset**.

Again, I'll plot against the true and predicted values:

```
[ ]: y_pred_cv = cross_val_predict(rfr, X_white, y_white, cv=cv)

plt.figure(figsize=(8, 6))
plt.scatter(y_white, y_pred_cv, color=palette_white[5], edgecolors="k",
    ↪label="Predicted vs Actual", alpha=0.7)
plt.plot([y_white.min(), y_white.max()], [y_white.min(), y_white.max()], "b--",
    ↪label="Perfect Prediction")
plt.xlabel("True Quality")
plt.ylabel("Predicted Quality")
plt.title("Task 4")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("../figures/task4_regression_white.png")
plt.show()
```



```
Exception ignored in: <function ResourceTracker.__del__ at 0x10aea8fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10a79cfe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10f79cfe0>
```

```

Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10d8a8fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x101fb0fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10ce94fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x106494fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
    File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked

```



```

ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x104fa8fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x1050a8fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10d3a4fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x104e94fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked
ChildProcessError: [Errno 10] No child processes
Exception ignored in: <function ResourceTracker.__del__ at 0x10d3a8fe0>
Traceback (most recent call last):
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 82, in __del__
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 91, in _stop
  File "/usr/local/Cellar/python@3.13/3.13.3/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in _stop_locked

```

```
s/3.13/lib/python3.13/multiprocessing/resource_tracker.py", line 116, in
_stop_locked
ChildProcessError: [Errno 10] No child processes
```

Observations The scatter plot above shows the predicted wine quality scores against the true labels for the white wine dataset, based on 10-fold cross-validated predictions from a Random Forest Regressor.

Compared to the red wine plot, the predictions for white wine are **generally close to the diagonal line**, indicating good performance. However, there is **slightly more vertical spread** than in the red wine plot, especially around quality scores of 6 to 8. This aligns with the metrics: the white wine model had a **higher RMSE** and a **lower R2** than red wine, meaning its predictions were slightly less precise and explained less variance. Despite this, overall, **white wine regression performs well**, though **red wine regression is slightly more accurate** and confident based on both metrics and visual patterning.

Summary In summary, to evaluate model performance, I used **classification** and **regression** approaches with **10 fold cross-validation** on *both* datasets, and compared them using appropriate metrics.

Binary Classification Binary classification was performed by splitting the **quality** label into **high** and **low** with the threshold **quality > 6** indicating **high** quality wines. Models were evaluated using **F1-score**, **AUC** and plotted **ROC curves**.

- **Red Wine**
 - F1-score: 0.459 +- 0.078
 - AUC: 0.935
- **White Wine**
 - F1-score: 0.539 +- 0.051
 - AUC: 0.890

The red wine classifier achieved a higher AUC, indicating stronger class separation, but the white wine model had a higher F1-score, suggesting it was more balanced in correctly identifying both classes.

Regression For the regression task, the goal was to predict the wine quality score as a continuous variable. Evaluation used the suggested regression metrics: **MSE**, **RMSE**, **MAE**, and **R2 scores**.

- **Red Wine**
 - MSE: 0.339 +- 0.038
 - RMSE: 0.581 +- 0.032
 - MAE: 0.433 +- 0.026
 - R2: 0.476 +- 0.070
- **White Wine**

- MSE: 0.438 +- 0.030
- RMSE: 0.662 +- 0.023
- MAE: 0.511 +- 0.013
- R2: 0.439 +- 0.040

The red wine regression model slightly outperformed the white wine model across all metrics, indicating more accurate predictions and better variance explanation. This is consistent with my performed visual analysis.

0.10 Extensions

Throughout this notebook, I have completed a majority of the listed extensions on Canvas. I'll provide references below so you may navigate to them easily:

1. Additional descriptive analysis

- I completed this in Task 1, exploratory analysis, under [extension 1](#). As well as this, much of my [the datasets at a glance](#) is extensive to the requirements and fits as additional descriptive analysis.
- 2. *“Variable selection for model learning”*
 - I completed this in Task 2, variable analysis, under [final feature sets](#) and used these in Task 3 modelling.

0.11 Conclusion

Throughout this notebook, I have explored the relationships between wine features and quality across both the red and white (and combined) datasets. I performed detailed exploratory analysis, visualised feature distributions, and engineered new variables such as `isSweet` and `alcohol_cat` to improve model performance.

I trained and evaluated models using two supervised learning approaches: **binary classification** and **regression**. For classification, I used multiple models, with repeated analysis performed on the **Random Forest** model. I also tested multiple thresholds (e.g. `high quality > 6`) and evaluated all models using a range of metrics; **F1-score**, **AUC**, and **plotted ROC curves**. For binary classification, I achieved the best balance on the *white wine* dataset.

For regression, I again mainly used the **Random Forest** regressor, though I compared many different models using different metrics again, this time **MSE**, **RMSE**, **MAE**, and **R2 scores**. Here, the red wine dataset had slightly stronger overall performance.

Models were properly evaluated using **10 fold cross-validation**, and final performance was visualised using ROC curves and prediction scatter plots. I also completed extension tasks through additional descriptive analysis and feature selection for my models.

In future, to improve performance further, I would: - Explore more advanced models (e.g. XGBoost)
- Perform extra parameter tuning using `GridSearchCV` on all models - Investigate better class balancing methods (as I suspect this was one of the biggest influences on my results).

0.12 References

This is not an exhaustive list of references: these are the most influential on this notebook. For a thorough list, see my Project Report.

1. [Randal Olson's Notebook](#)
2. [Chicago Crime Notebooks GitHub](#)
3. [Data Cleaning using Python](#)
4. [Markdown Cheat Sheet](#)
5. [Markdown Tables Help](#)
6. [Vinho Verde White Wine Quality Analysis](#)
7. [Vinho Verde Red Wine Quality Analysis](#)
8. Logistic Regression, Linear SVM, and Decision Trees
9. Non-Linearly Separable Data
10. Effects of Class Imbalance
11. Predicting Academic Performance